Preface

Matching theory is one of the central topics in graph theory and combinatorial optimization.

Matchings appear in a wide range of problems in computer science, mathematics, economics, and operations research. It provides a natural way to model situations in which agents, objects, or resources must be paired under certain restrictions.

Consider the case where there are n teachers and we have to decide which schools to assign them to based on some criteria. Criteria may vary; for example, each teacher may have preferences over the available schools, while schools may have specific ranking criteria based on their teaching needs. Similarly, n applicants may have preferences over houses they want to buy. There is only one side with preferences, but we still have two sides we want to match. An example with more than two sides is when we have a set of professors, projects, and students, and our purpose is to assign each project to a professor and then assign students to each project. Here, each assignment is a triplet, and there could be many criteria.

The algorithmic study of matchings has played an important role in the development of efficient combinatorial algorithms. Some matching problems admit elegant, strongly polynomial algorithms, while others become computationally difficult even with only small changes to the model.

The profound impact of matching theory on real-world economics and society was notably recognized in 2012, when Alvin E. Roth and Lloyd S. Shapley were awarded the Nobel Prize in Economic Sciences. Their recognition was rooted in the theory of stable allocations, which originated in the foundational 1962 work by David Gale and Lloyd Shapley.

Linear and integer programming provide another fundamental approach to optimization. The overwhelming majority of problems in Economics, Engineering, and many in Computer Science have linear functions as a basic characteristic. Consequently, Linear Programming has numerous applications across these areas, making it one of the most widely applied branches of Mathematics. It is a relatively new science, born to meet the demands of the Second World War. During the early years of its development, the simplex algorithm dominated. However, in subsequent decades, interior-point and exterior-point algorithms were developed, revolutionizing the landscape. These newer methods are often considered superior, particularly for large-scale optimization problems, as they offer polynomial-time complexity guarantees and can navigate the feasible region to reach the optimal solution much more efficiently in practice.

In integer programming, many natural decision variables are restricted to take values of zero or one, representing whether an object is selected, an edge is used, or an assignment is made. However, solving integer programs directly is generally difficult. A common strategy is therefore to relax the integrality restrictions and study the corresponding linear program. When the relaxation has integral optimal solutions, the original discrete problem can be solved through linear programming.

The purpose of this thesis is to study matching problems on bipartite graphs from

both of these perspectives. We first present classical algorithmic results for two fundamental problems in matching theory and then study related formulations through linear and integer programming. The final part of the thesis focuses on a Pareto-optimal matching problem in a one-to-one house-allocation model, where the behavior of the linear relaxation is substantially more complicated.

Chapter 2 is devoted to matching problems and algorithms. We begin with the maximum bipartite matching problem. After introducing the basic definitions, we present a classical augmenting-path implementation with running time $O(nm)$, where n denotes the number of vertices and m the number of edges. We then give an implementation of the Hopcroft–Karp algorithm, which improves the running time to $O(\sqrt{n}(n+m))$ by finding many shortest augmenting paths in phases. The chapter continues with the stable marriage problem and the Gale–Shapley algorithm, one of the most influential algorithms in matching theory.

A particular emphasis of this chapter is the presentation of the algorithms. The pseudocodes are written in a detailed form, so that their translation into an actual programming language is almost immediate. Moreover, the proofs are adapted to the implementations rather than being presented only at a high conceptual level. This makes the connection between the code, the algorithmic idea and the correctness argument explicit. The goal is not only to state that the algorithms work, but also to make clear why each step is necessary.

Chapter 3 introduces the polyhedral point of view. We first recall the basic language of linear programming, integer programming, polyhedra, polytopes, and integrality. We then return to the matching problems studied in Chapter 2 and examine them through linear and integer programming formulations. For the maximum bipartite matching problem, the linear relaxation captures the integer structure of the problem, and optimal matching solutions can be obtained through linear programming. A similar phenomenon appears in the stable marriage problem, where the corresponding stable matching polytope has an integral linear description. These examples illustrate how polyhedral structure can explain why certain discrete optimization problems admit efficient linear programming approaches.

Chapter 4 studies a different problem: Pareto-optimal matchings in a one-to-one house allocation model. In this setting, agents have strict preference lists over houses, and the aim is to describe Pareto-optimal matchings. We present an integer programming formulation for this problem and then study its linear relaxation. Unlike the classic examples in Chapter 3, the relaxation is not generally integral. We demonstrate this by constructing fractional solutions that satisfy the relaxation but do not correspond to convex combinations of Pareto optimal matching vectors.

A further difficulty is that one family of constraints in the formulation can be very large. To handle this, we use a separation approach. So, instead of listing all such constraints in advance, we search for violated constraints only when they are needed. This allows the relaxation to be solved more efficiently in practice and enables computational experiments on small instances.

The computational part of the thesis compares the linear relaxation with the actual convex hull of Pareto optimal matching vectors. For several small instances, we enumerate Pareto optimal matchings, compute the corresponding convex hull, and inspect its linear description. This allows us to identify inequalities that are valid for the true Pareto optimal matching polytope but are not enforced by the original relaxation. Based on these experiments, we propose and prove two families of additional valid inequalities: prefix-cover inequalities and safe-holder inequalities. These constraints strengthen the relaxation by excluding fractional solutions that do not lie within the convex hull of Pareto-optimal

matchings.

The thesis therefore follows a progression from algorithms to polyhedra and then to computational polyhedral analysis. The first part explains in detail how classical matching algorithms work. The second part shows how linear programming can exactly capture some matching problems. The final part shows that for Pareto-optimal house allocation, the situation is not that simple. The natural relaxation is not sufficient, and additional valid inequalities are needed to obtain a tighter description. The implementation details and source code are included in the appendices, so that the computational experiments can be reproduced and extended.

We assume a basic background in subjects typically covered in an undergraduate curriculum, such as graph theory, algorithms, computational complexity, and linear algebra. Familiarity with these areas is sufficient to follow the material presented in this thesis.

Contents

Chapter 1

Matching Problems and Algorithms

1.1 Definitions

Let $G = (V, E)$ be a simple (without loops and parallel edges) undirected graph with $|V| = n$ vertices and $|E| = m$ edges. We call G bipartite if we can partition its vertex set into two sets V_1, V_2 such that for each edge $\{u, v\} \in E$, one endpoint belongs to V_1 and the other to V_2. If every vertex of V_1 is adjacent to every vertex of V_2 and $|V_1| = n_1, |V_2| = n_2$, then we call G a complete bipartite graph and denote it by K_{n_1,n_2}. By a path, we mean a simple path, i.e., a path in which no vertex is repeated. The length of a path is the number of edges it contains.

From set theory, we recall that the symmetric difference of two sets A and B, denoted by $A \triangle B$, is the set of elements that belong to A or B but not both. Thus, $A \triangle B = (A \backslash B) \cup (B \backslash A) = (A \cup B) \backslash (A \cap B)$. Also, if we want to add an element x to set A, we will use the shorthand $A + x \coloneqq A \cup \{x\}$, and if we want to remove it $A - x \coloneqq A \backslash \{x\}$.

For non-negative functions T and f, we write $T(n) = O(f(n))$ if and only if there exist positive constants c and n_0 such that $T(n) \leqslant cf(n)$ for all $n \geqslant n_0$. Most of the time, $T(n)$ will denote a bound on the worst-case running time of an algorithm defined on the positive integers. Similarly, we say that $T(n) = \Omega(f(n))$ if and only if there exist positive constants c and n_0 such that $T(n) \geqslant cf(n)$ for all $n \geqslant n_0$. Finally, we say that $T(n) = \Theta(f(n))$ if and only if $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$.

A matching M of a graph $G = (V, E)$ is a subset of E such that each vertex is incident to at most one edge of the subset. The cardinality of a matching is the number of edges it contains. A vertex is called matched if it is incident to an edge of M. Otherwise, it is called unmatched or free. A matching is called maximal if no edge from $E \backslash M$ can be added without violating the matching property. A matching is maximum if no other matching has more edges. Clearly, every maximum matching is maximal (the converse is not always true). The following two definitions are fundamental in the study of matchings.

Definition 1.1.1. *A path P in G with respect to a matching M is called alternating if its edges are alternatingly out of and in M.*

Definition 1.1.2. *A path P in G with respect to a matching M is called augmenting if it has positive length, is alternating, and its endpoints are free.*

It is straightforward to see that every augmenting path has odd length. We will use the term M-augmenting whenever we want to emphasize the matching M.

1.2 Maximum Bipartite Matching

In this section, we present two famous algorithms for finding a maximum bipartite matching and prove their correctness.

The first algorithm is derived from a theorem of Berge [Ber57], previously observed by Petersen [Pet91].

Theorem 1.2.1. *Let $G = (V, E)$ be a graph and let M be a matching in G. Then M is maximum if and only if there is no M-augmenting path in G.*

Proof. Suppose that M is maximum. For the sake of contradiction, assume that there exists an M-augmenting path P with u and v as its endpoints in G. Then we can see that the matching $M' = M \triangle E(P)$ is a valid matching in G and satisfies $|M'| = |M| + 1$. Indeed, if we remove all edges of P that belong to M and add the remaining ones to M, we add one more edge to M in total (because P has odd length). The new matching is valid because u and v were free, and they were not incident to any matching edge, and every internal vertex of P remains incident to exactly one edge of M'. All vertices outside P are unaffected. As a result, the new matching M' is valid and its cardinality is greater than that of M. This is a contradiction.

Now suppose that G does not contain any M-augmenting path and that a matching M' exists in G such that $|M'| > |M|$. Consider the subgraph $G_0 = (V, M_0)$ where $M_0 = M \triangle M'$. Each vertex in G_0 may be matched in both matchings or in either one. So, every vertex has degree at most 2. Hence, every component of G_0 is either a path or a cycle of even length (otherwise, there would exist a vertex that is incident to two edges in the same matching). Because M_0 has more edges of M' than of M, this also means that there exists a path P with more edges of M' than of M. Necessarily, the edges of this path are alternating (otherwise a vertex would be incident to more than one edge of the same matching) and its endpoints are free with respect to M. Hence, P is an M-augmenting path, which contradicts our assumption that G does not contain any M-augmenting path. □

1.2.1 Augmenting-Path Algorithm

Based on the theorem above, we can write an algorithm that will try to find an augmenting path, alternate its edges (this will increase the size of the current matching by one), and repeat the whole process until we can't find any augmenting path. $N(v)$ represents all the vertices u such that $\{v, u\} \in E(G)$.

Algorithm 1.1 Augmenting Path Algorithm

Require: Graph $G = (V, E)$, vertex v
Ensure: Array matched such that $matched[u] = v$ and $u \in V_2$ and $v \in V_1$ if and only if $\{u, v\} \in E(G)$ belongs to the matching

```
1: function DfsAugmentingPath(int v, bool visited[], int matched[])
2:     if visited[v] then
3:         return FALSE;
4:     end if
5:     visited[v] ←TRUE;
6:     for all u ∈ N(v) do
7:         if matched[u] = ∅ or DfsAugmentingPath(matched[u], visited, matched) then
8:             matched[u] ← v;
9:             return TRUE;
10:        end if
11:    end for
12:    return FALSE;
13: end function
```

Algorithm 1.1 is getting called from all vertices in the left part of bipartite graph (let's call it V_1). We will use an extra array called *visited* to track the vertices from V_1 that we have already visited across the path and we have to reset it to be false for all these vertices before the initial call. Array matched marks the matched vertices only from the right part of the bipartite graph (let's call it V_2). To get the edges of the matching, we simply take all the edges $\{v, matched[v]\}$ such that $matched[v]$ is not empty and store them to set *matching*.

Algorithm 1.2 Maximum Bipartite Matching in $O(nm)$

Require: Bipartite graph $G = (V_1 \cup V_2, E)$
Ensure: Set *matching* that corresponds to the edges of the maximum matching

```
1: function MainAugmentingPath(Graph G, set matching)
2:     for all v ∈ V2 do
3:         matched[v] ← ∅;
4:     end for
5:     for all v ∈ V1 do
6:         for all u ∈ V1 do
7:             visited[u] ← FALSE;
8:         end for
9:         DfsAugmentingPath(v, visited, matched);
10:    end for
11:    matching ← ∅;
12:    for all v ∈ V2 do
13:        if matched[v] ≠ ∅ then
14:            matching ← matching ∪ {{matched[v], v}};
15:        end if
16:    end for
17:    return matching;
18: end function
```

The idea is that if the initial call is from a free vertex, say v, and we cannot find another

free vertex u such that the edge $\{v, u\}$ exists, then we simply try to find a path (v, x, y) such that $\{v, x\}$ is not in the matching, but $\{x, y\}$ is, so we can start trying again from the vertex y, until we find a free vertex and then recursively alternate the path.

Lemma 1.2.2. *An initial call to Algorithm 1.1 from a free vertex $v \in V_1$ (assuming that visited$[w]$ = FALSE for every $w \in V_1$ before the call) returns* TRUE *if and only if there exists an augmenting path with endpoint v. In this case, it alternates the edges of such a path.*

Proof. Suppose that a free vertex $u \in V_2$ exists such that $\{v, u\} \in E(G)$. Then from line 7 the algorithm successfully finds an augmenting path. Now suppose that such a vertex doesn't exist and $P = (v, v_1, v_2, ..., u)$ any augmenting path with the vertex v as its endpoint. Since P is alternating, the edge $\{v, v_1\}$ does not belong to the matching, while $\{v_1, v_2\}$ does. Because the algorithm checks all neighbors of v (line 6), it will eventually examine v_1 (assuming that all the other neighbors, if they exist, return false), and line 7 does a recursive call for $matched[v_1] = v_2$ (note that v_2 is impossible to be visited, otherwise an alternative path $(v, ..., v_2)$ exists and the algorithm would have already found another augmenting path from that recursive call to u or to another free vertex). Repeating the same argument along P, the algorithm eventually reaches the free vertex $u \in V_2$, where line 7 is true since $matched[u] = \varnothing$. The recursive calls then return TRUE and update the array matched accordingly, thus alternating the edges of P.

Conversely, if the algorithm returns TRUE, then this can only happen because it reaches a free vertex of V_2, either directly or through recursive calls. Tracing these successful recursive calls backwards gives an alternating path from v to a free vertex of V_2, and the updates of *matched* alternate the edges of this path. □

From this, we get the following remark.

Remark 1.2.3. *After an initial call of Algorithm 1.1 from a free vertex $v \in V_1$, the only vertices that can become matched are v and one free vertex of V_2. If no augmenting path is found, then the matching remains unchanged.*

Then, we can prove the following lemma.

Lemma 1.2.4. *Every initial call to Algorithm 1.1 from Algorithm 1.2 takes a free vertex as parameter.*

Proof. This is obviously true in the first iteration. Assume that we are at iteration i, and that the statement holds for all previous iterations. Let v be the vertex from which we are about to initially call Algorithm 1.1. If v is free, then we are done. If it is not, v must have become matched during some previous iteration. However, by Remark 1.2.3, in each iteration the only vertex of V_1 that can become matched is the vertex from which Algorithm 1.1 was initially called. Since Algorithm 1.1 has never been initially called from v before iteration i, the vertex v could not have become matched earlier. □

The output of the Algorithm 1.2 (set matching) is always a valid matching. Since the algorithm initializes with an empty matching and only updates it by alternating edges along valid augmenting paths starting from free vertices (Lemma 1.2.2), the maintained set of edges is always a valid matching. Now we have to prove that it is maximum.

Theorem 1.2.5. *Algorithm 1.2 returns a maximum matching.*

Proof. Let $v_1, v_2, \ldots, v_k$ be the vertices of V_1 in the order in which Algorithm 1.2 processes them. For each $i \in \{0, 1, \ldots, k\}$, let M_i be the matching after the first i iterations of the algorithm, with $M_0 = \varnothing$. Also, let G_i be the subgraph of G induced by the vertex set $\{v_1, v_2, \ldots, v_i\} \cup V_2$. We will prove by induction on i that M_i is a maximum matching of G_i.

For $i = 0$, the graph G_0 has no vertices in V_1, so the empty matching M_0 is trivially maximum.

Now assume that M_i is a maximum matching of G_i, and consider iteration $i+1$, where the algorithm processes the vertex v_{i+1}. By Lemma 1.2.4, the initial call to Algorithm 1.1 is made from a free vertex. We consider two cases.

Case 1: *DfsAugmentingPath*(v_{i+1}) (Algorithm 1.1 with parameter the vertex v_{i+1}) returns TRUE. Then, by Lemma 1.2.2, the algorithm finds an augmenting path in G_{i+1} with endpoint v_{i+1} and alternates its edges. Therefore,

$$|M_{i+1}| = |M_i| + 1.$$

On the other hand, any matching of G_{i+1} can use at most one edge incident to the new vertex v_{i+1}. Hence, by removing that edge if it exists, we obtain a matching of G_i, and therefore every matching of G_{i+1} has size at most $|M_i| + 1$. Since M_i is maximum in G_i, it follows that M_{i+1} is a maximum matching of G_{i+1}.

Case 2: *DfsAugmentingPath*(v_{i+1}) returns FALSE. Then, again by Lemma 1.2.2, there is no M_i-augmenting path in G_{i+1} with endpoint v_{i+1}. Since M_i is maximum in G_i, Theorem 1.2.1 implies that there is no M_i-augmenting path contained entirely in G_i. Thus, there is no M_i-augmenting path in G_{i+1} at all, because any augmenting path in G_{i+1} that is not contained in G_i must involve the only new vertex of V_1, namely v_{i+1}. Therefore, by Theorem 1.2.1, M_i is already a maximum matching of G_{i+1}. Since the matching does not change in this case, we have

$$M_{i+1} = M_i,$$

and so M_{i+1} is a maximum matching of G_{i+1}.

In both cases, M_{i+1} is a maximum matching of G_{i+1}. This completes the induction. □

Before each search, we initialize the *visited* array which takes $O(n)$ time. We call *DfsAugmentingPath* for each vertex $v \in V_1$ searching for an augmenting path. To find an augmenting path, we will visit each vertex and each edge at most once (similar to the classic DFS), so the total complexity is $O(nm)$ when $m = \Omega(n)$.

1.2.2 Hopcroft-Karp Algorithm

Hopcroft and Karp provided a faster algorithm [HK73], again based on finding augmenting paths, but in a smarter way. The algorithm runs in phases. In each phase, we find a maximal set of vertex-disjoint shortest M-augmenting paths and take the symmetric difference of the current matching M with the union of these paths. The process stops when no augmenting path can be found.

Our implementation is based on the version of Shimon Even and Alon Itai, who made significant modifications to Dinic's algorithm for the maximum flow problem [Din70, Din06]. As we will see at the end of this section, maximum flow and maximum bipartite matching are closely connected problems.

The key idea of the algorithm is to efficiently find these paths and alternate their edges. To do that, we construct a new graph called the layered network. We describe this graph with an array called *level*, defined on the vertices of V_1. Initially, every free vertex of V_1 gets level 1. Then, if a vertex $u \in V_1$ has level i, every vertex of V_1 that can be reached from u by first taking an unmatched edge to a vertex of V_2 and then a matched edge back to V_1 gets level $i + 1$, provided it has not already received a level. The vertices are visited in Breadth-First Search (BFS) order, starting from all free vertices of V_1. In this way, $level[v]$ represents the minimum alternating distance of $v \in V_1$ from the set of free vertices of V_1 in the current phase. The edges in the layered network are directed according to the BFS levels, so the resulting graph is a directed acyclic graph (DAG).

Algorithm 1.3 BFS Layered Network Construction

Require: Bipartite graph $G = (V_1 \cup V_2, E)$, arrays *next*, *prev* represent the current matching
Ensure: Array *level* for the vertices of V_1 describes the layered network

```
1: function BfsLayeredNetwork(int next[], int prev[], int level[])
2:     treeDepth ← 0;
3:     initialize empty queue Q;
4:     for all u ∈ V1 do
5:         if next[u] = ∅ then
6:             level[u] ← 1;
7:             Q.push(u);
8:         end if
9:     end for
10:    while not Q.isEmpty() do
11:        u ← Q.pop();
12:        for all x ∈ N(u) do
13:            if prev[x] = ∅ then
14:                treeDepth ← level[u];
15:                continue;
16:            end if
17:            if prev[x] = u then
18:                continue;
19:            end if
20:            if level[prev[x]] = 0 then
21:                level[prev[x]] ← level[u] + 1;
22:                Q.push(prev[x]);
23:            end if
24:        end for
25:        if treeDepth ≠ 0 then
26:            break;
27:        end if
28:    end while
29:    for all u ∈ V1 do
30:        if level[u] > treeDepth then
31:            level[u] ← 0;
32:        end if
33:    end for
34:    return (treeDepth ≠ 0);
35: end function
```

The first time we encounter a free vertex in V_2, we determine the last useful level in the current phase and we store it in the variable *treeDepth*. From that, we can also determine the length of a shortest augmenting path in the current phase (which is $2 \cdot \mathit{treeDepth} - 1$). We do not need to continue the BFS deeper than that level and the vertices that remain in the queue belong either to the same level or to the next one. The vertices of the same level may still be used in the searching step to find other shortest augmenting paths, since we have already assigned their correct level. On the other hand, vertices that have already been assigned level $\mathit{treeDepth} + 1$ correspond to longer paths, so we reset their level to 0.

We define an array *next* for every $v \in V_1$ such that $\mathit{next}[v] = u$ if and only if the

edge $\{v, u\}$ belongs to the current matching; otherwise, $next[v] = \varnothing$. Similarly, for every $u \in V_2$, we define $prev[u] = v$ if and only if the edge $\{u, v\}$ belongs to the current matching; otherwise, $prev[u] = \varnothing$. Thus, the arrays *next* and *prev* represent the current matching and are updated whenever we find an augmenting path and alternate its edges.

Theorem 1.2.6. *Algorithm 1.3 correctly computes the array level of the layered network of the current matching M. More precisely, if it returns* FALSE*, then no M-augmenting path exists in G. Otherwise, for every vertex $v \in V_1$, we have $level[v] \neq 0$ if and only if there exists an M-alternating path from a free vertex of V_1 to v. Moreover, if $level[v] \neq 0$, then the length of a shortest such path is $2 \cdot level[v] - 2$.*

Proof. For every free vertex $v \in V_1$, the algorithm sets $level[v] = 1$, and the corresponding alternating path has length 0. Now suppose that a vertex $u \in V_1$ is popped from the queue and that the statement already holds for u. Let $x \in V_2$ be a neighbor of u.

If $prev[x] = \varnothing$, then extending a shortest alternating path to u by the edge $\{u, x\}$ gives an M-augmenting path of length $2 \cdot level[u] - 1$. If $prev[x] = u$, then $\{u, x\}$ is a matched edge and cannot be used to continue an alternating path. Otherwise, if $prev[x] = w \neq u$ and $level[w] = 0$, then by appending the unmatched edge $\{u, x\}$ and the matched edge $\{x, w\}$ to a shortest alternating path ending at u, we obtain an alternating path from a free vertex of V_1 to w of length $(2 \cdot level[u] - 2) + 2 = 2 \cdot (level[u] + 1) - 2$. Hence, the algorithm correctly sets $level[w] = level[u] + 1$. Since vertices are processed in BFS order, this is the minimum possible alternating distance to w.

Therefore, for every vertex $v \in V_1$ with $level[v] \neq 0$, the value $level[v]$ is exactly the level of v in the layered network, and the length of a shortest alternating path from a free vertex of V_1 to v is $2 \cdot level[v] - 2$.

Finally, if the algorithm returns FALSE, then no free vertex of V_2 is reached during the BFS. Therefore, there is no M-augmenting path in G. Indeed, if such a path existed, then the vertex just before its free endpoint on a shortest such path would lie in the layered network, and the free vertex in V_2 would also be reached. □

Corollary 1.2.7. *If Algorithm 1.3 returns* TRUE*, then the length of a shortest M-augmenting path is $2 \cdot treeDepth - 1$.*

The searching step, Algorithm 1.4, is almost the same as Algorithm 1.1, so we will omit the proof. This time, we move across the layered network based on the *level* array. We also use an array *ptr* for all vertices in V_1. This array holds for each vertex a counter that indicates the first neighbor that has not been checked yet. Hence, if we return to the same vertex in the same phase, we do not examine again the neighbors that have already returned FALSE. In particular, if $ptr[u]$ reaches the end of the adjacency list of u, then no augmenting path can be found through u in the current phase.

We could of course combine the two *if* statements into one.

Algorithm 1.4 DFS Phase of Hopcroft-Karp

Require: Bipartite graph $G = (V_1 \cup V_2, E)$, vertex $u \in V_1$, arrays *next*, *prev*, *level*, *ptr*
Ensure: Returns TRUE if a shortest augmenting path is found from u

```
1: function DfsHopcroftKarp(int u, int next[], int prev[], int level[], int ptr[])
2:     for ptr[u] to |N(u)| − 1 do
3:         x ← the ptr[u]-th neighbor of u;
4:         if prev[x] = ∅ then
5:             prev[x] ← u;
6:             next[u] ← x;
7:             ptr[u] ← |N(u)|;
8:             return TRUE;
9:         end if
10:        if level[prev[x]] = level[u] + 1 and DfsHopcroftKarp(prev[x], next, prev, level,
   ptr) then
11:            prev[x] ← u;
12:            next[u] ← x;
13:            ptr[u] ← |N(u)|;
14:            return TRUE;
15:        end if
16:    end for
17:    return FALSE;
18: end function
```

Algorithm 1.5 and lines 9-21 run the phases of the algorithm. In each phase, we call Algorithm 1.3 (line 9) to construct the layered network, find a maximal set of vertex-disjoint shortest augmenting paths in this graph by calling Algorithm 1.4 for each free vertex of V_1 (lines 13-17), and repeat the whole process. Note that we have to initialize the *ptr* array to 0 before starting the search, because all vertices will check their first neighbor first. We also have to initialize the *level* array to zero before building the layered network. The set *matching* contains the edges of the maximum matching.

Algorithm 1.5 Hopcroft–Karp Algorithm

Require: Bipartite graph $G = (V_1 \cup V_2, E)$
Ensure: Array *matching* contains the edges of the maximum matching

```
1: function HopcroftKarp( )
2:     for all u ∈ V1 do
3:         next[u] ← ∅;
4:         level[u] ← 0;
5:     end for
6:     for all x ∈ V2 do
7:         prev[x] ← ∅;
8:     end for
9:     while BfsLayeredNetwork(next, prev, level) do
10:        for all u ∈ V1 do
11:            ptr[u] ← 0;
12:        end for
13:        for all u ∈ V1 do
14:            if next[u] = ∅ then
15:                DfsHopcroftKarp(u, next, prev, level, ptr);
16:            end if
17:        end for
18:        for all u ∈ V1 do
19:            level[u] ← 0;
20:        end for
21:    end while
22:    matching ← ∅;
23:    for all u ∈ V1 do
24:        if next[u] ≠ ∅ then
25:            matching ← matching ∪ {{u, next[u]}};
26:        end if
27:    end for
28:    return matching;
29: end function
```

The figure below illustrates a possible instance of a phase of the layered network.

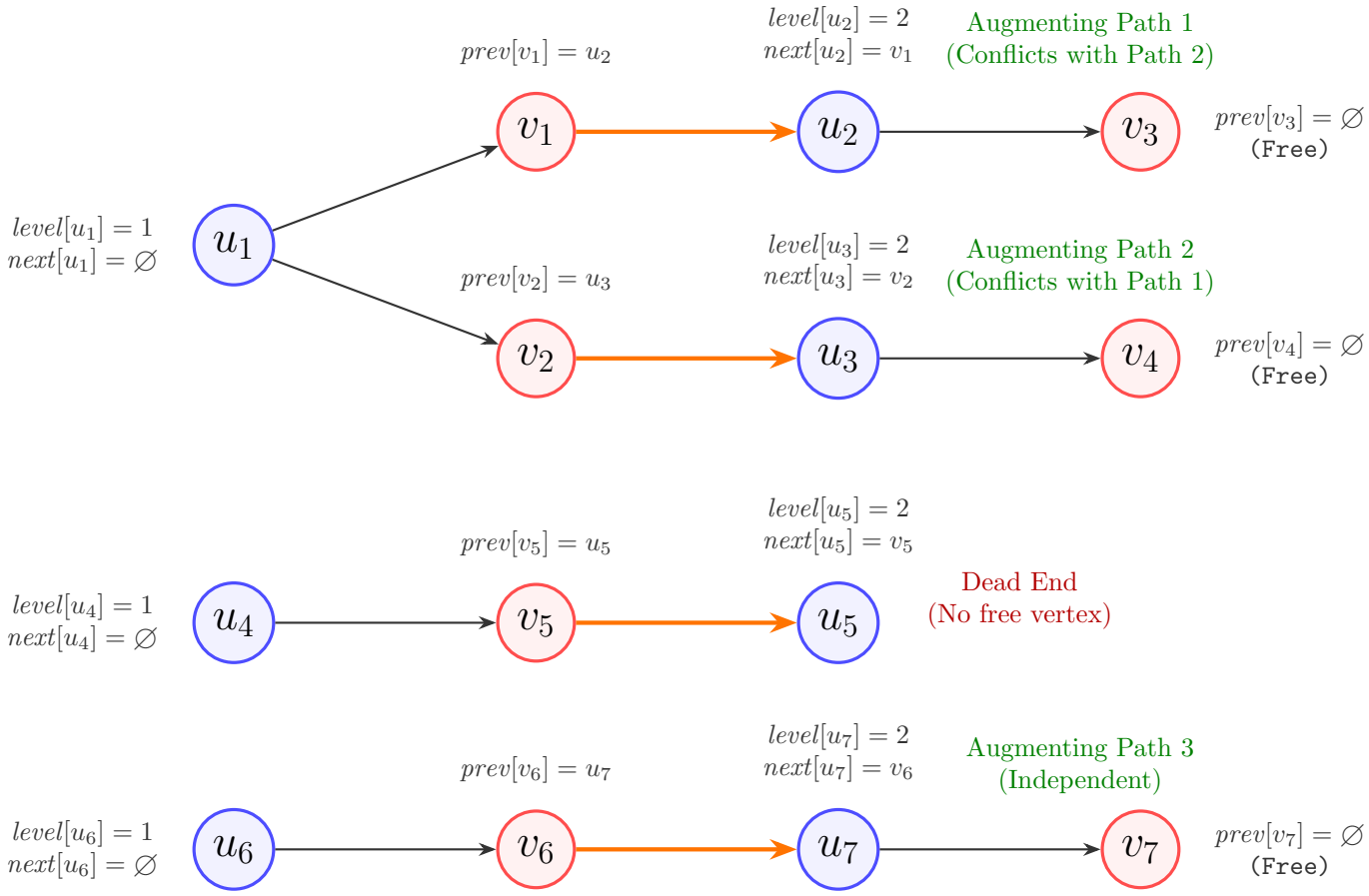

Figure 1.1: A phase of the Hopcroft-Karp algorithm. The BFS starts from free vertices $u_1, u_4, u_6 \in V_1$. The layered network reveals three augmenting paths of length 3. However, the paths ending at v_3 and v_4 share the vertex u_1, so only one can be selected in the maximal vertex-disjoint set. The path $u_6 - v_6 - u_7 - v_7$ is vertex-disjoint from the others and will be included alongside whichever path is chosen from u_1. The path starting at u_4 is a dead end.

It is evident that constructing the layered network takes no more than $O(n + m)$. Moreover, lines 13-17 take no more than $O(n + m)$ as each edge is examined at most once (with the help of *ptr* array) and as a result when vertex returns FALSE, we are not going to process it again in the current phase.

Lemma 1.2.8. *Each phase, that is calculating the maximal set of vertex-disjoint shortest M-augmenting paths and taking the symmetric difference with current matching M takes no more than $O(n + m)$ time.*

We shall show that we need at most $O(\sqrt{n})$ phases, but before that, we have to prove some important lemmas.

Lemma 1.2.9. *Let M and M' be two matchings of a graph G, with $|M'| > |M|$. Then the graph $M \triangle M'$ contains at least $|M'| - |M|$ vertex-disjoint M-augmenting paths.*

Proof. We follow the same logic as in Theorem 1.2.1. We consider the graph $G = (V, M^*)$ with $M^* = M \triangle M'$. We can see that the graph contains components, each of which is an alternating path or an even-length cycle. Cycles have the same number of edges from M and M', and the same holds for alternating paths of even length. Each component that

represents an augmenting path has one more edge from one matching than from the other. In fact, every M-augmenting path has one more edge from $M^{'}$ than M. From $|M^{'}| > |M|$, we can see that we have more M-augmenting paths than $M^{'}$-augmenting paths, and there must be at least $|M^{'}| - |M|$ such paths. This completes the proof. □

The following is a special case of Lemma 1.2.9.

Lemma 1.2.10. *Given a matching M and a maximum matching M^*, the graph contains $|M^*| - |M|$ vertex-disjoint M-augmenting paths.*

Lemma 1.2.11. *Let $P = \{P_1, P_2, \cdots, P_t\}$ be a maximal set of vertex-disjoint shortest M-augmenting paths with length x obtained during a single phase of Algorithm 1.5 and $M^{'} = M \triangle (P_1 \cup P_2 \cup \cdots \cup P_t) = M \triangle P_1 \triangle P_2 \triangle \cdots \triangle P_t$ and let Q be an $M^{'}$-augmenting path. The set $R = (P_1 \cup P_2 \cup \cdots \cup P_t) \triangle Q$ has at least $x(t+1)$ edges.*

Proof. From $M^{'} = M \triangle (P_1 \cup P_2 \cup \cdots \cup P_t)$, we have

$$P_1 \cup P_2 \cup \cdots \cup P_t = M \triangle M^{'}$$

That is,

$$R = (M \triangle M^{'}) \triangle Q = M \triangle (M^{'} \triangle Q)$$

Since Q is an $M^{'}$-augmenting path, $M^{'} \triangle Q$ (which is obviously a valid matching) has size of $|M^{'}| + 1 = |M| + t + 1$ (plus $t + 1$ because it will get $t + 1$ more edges from alternating the paths of P and the path Q). By Lemma 1.2.9, there is at least $|M^{'} \triangle Q| - |M| = t + 1$ M-augmenting paths. Because every M-augmenting path has length at least x, it follows that R has at least $x(t + 1)$ edges. □

Lemma 1.2.12. *The length of the shortest augmenting path (if exists) increases in every phase.*

Proof. Let's assume that the current matching is M and denote by $P = \{P_1, P_2, \cdots, P_t\}$ the maximal set of vertex-disjoint shortest M-augmenting paths of length x in one phase of Algorithm 1.5. Moreover, suppose that we take $M^{'} = M \triangle (P_1 \cup P_2 \cup \cdots \cup P_t)$ (the matching that we get at the end of phase) and let Q be an $M^{'}$-augmenting path. Obviously, Q has a common vertex v with at least one path from P, otherwise, Q would also be an M-augmenting path and either $|Q| > x$ (in this case we are done) or it would contradict the maximality of the set. Let $R = (P_1 \cup P_2 \cup \cdots \cup P_t) \triangle Q$. From Lemma 1.2.11, R has at least $x(t + 1)$ edges. Let $P_i \in P$ such that $v \in P_i$. We observe that v is incident with a matched edge in $M^{'}$ because at the end of the phase we alternate the edges of each path in P. The endpoints of Q do not belong to $M^{'}$ because they are free, so it is guaranteed that v is an internal vertex of Q and therefore it is also incident with a matched edge in $M^{'}$. Obviously, it's impossible for v to be incident with two different matched edges from $M^{'}$, so P_i and Q have at least one edge from $M^{'}$ in common. It follows that

$$xt + x \leqslant |R| \leqslant |P_1 \cup P_2 \cup \cdots \cup P_t \cup Q| \leqslant xt + |Q| - 1$$

and hence $x + 1 \leqslant |Q|$. □

Lemma 1.2.13. *Algorithm 1.5 performs at most $O(\sqrt{n})$ phases.*

Proof. Assume that we have already performed $\sqrt{n}$ phases and the current matching is M. From Lemma 1.2.10, we know that there exist at least $|M^*| - |M|$ vertex-disjoint M-augmenting paths for a maximum matching M^*. From Lemma 1.2.12 all these augmenting paths will have length at least $\sqrt{n}$, so there cannot be more than $\sqrt{n}$ (we would need more than n vertices) of them, which means that $|M^*| - |M| \leqslant \sqrt{n}$. It follows from this that we can't perform more than $\sqrt{n}$ phases from that point. We conclude that the total phases can't be more than $2\sqrt{n} = O(\sqrt{n})$. □

From Lemma 1.2.8 and Lemma 1.2.13 we get the complexity of Hopcroft-Karp algorithm.

Theorem 1.2.14. *Algorithm 1.5 returns a maximum matching in $O((n+m)\sqrt{n})$ time.*

Maximum Bipartite Matching from Max-Flows. We can reduce the bipartite maximum matching problem to the maximum flow problem by adding two extra nodes to the initial graph. The first vertex is called the source and will point to all vertices in V_1 with directed edges, and the second vertex is called the sink, with all vertices from V_2 pointing to it. Moreover, all the edges of the initial graph will turn into directed edges, pointing to V_2 from a vertex of V_1. Assuming capacities of one for each edge (so there exists an integral maximum flow, that is every edge will have flow of 0 or 1), the maximum flow from source to sink is equal to the maximum bipartite matching. Actually, this is easy to see, because all paths that carry flow from the source to the sink should be edge-disjoint and use exactly one edge from V_1 to V_2, while the flow passing through each vertex of these paths is exactly one (which means that no vertex is repeated). We can get a maximum flow in the resulting graph from a maximum matching in the initial graph by just taking all the edges of matching and picking the edges from the source to the corresponding V_1 vertices, and the edges from the corresponding V_2 vertices to the sink.

Consequently, someone could make the above reduction and use a famous flow algorithm, like Dinic [Din70] to solve the maximum bipartite problem with $O(n^2m)$. It turns out, that using some kind of the arguments we talked before, Dinic's algorithm will perform with the same complexity as Hopcroft-Karp algorithm.

Recent Research. Maximum flow is well-known problem in computer science and has been studied extensively over the years. Finding faster algorithms for the maximum flow problem, and especially for the minimum-cost flow problem (to which maximum flow can be reduced), could lead to breakthroughs in many areas. For a long time, algorithms based primarily on combinatorial methods were far from the almost-linear-time bounds achieved by more recent continuous techniques. In 2023, a deterministic $m^{1+o(1)}$-time algorithm for exact maximum flow and minimum-cost flow on directed graphs with polynomially bounded integral demands, costs, and capacities was discovered [BCK+23]. This line of work also implies an $m^{1+o(1)}$-time algorithm for the maximum bipartite matching problem. On the other hand, randomized combinatorial algorithms were later obtained independently for exact maximum flow [BBST24] and for maximum bipartite matching [CK24].

1.3 The Stable Marriage Problem

Assume that we are given a set of n men $\{m_1, m_2, \ldots, m_n\}$ and n women $\{w_1, w_2, \ldots, w_n\}$, where each man has a strict preference ranking over all n women, and each woman also has a strict preference ranking over all n men. We say that a *matching* is a set of unordered pairs consisting of a man and a woman (similarly to the previous section, where we considered the undirected edges of a bipartite graph), where each man and each woman appears in at most one pair. A *marriage* is a perfect matching between men and women, meaning that every man and every woman is matched exactly once. A marriage is *unstable* if there exists a pair $\{m_i, w_j\}$ that does not belong to the matching such that m_i prefers w_j over his actual partner, and w_j prefers m_i over her actual partner. We call such a pair a *blocking pair*. Otherwise, we say that it is *stable*.

The following example shows a stable marriage.

Example 1.3.1. *Consider an instance of four men*

$$\{m_1, m_2, m_3, m_4\}$$

and four women

$$\{w_1, w_2, w_3, w_4\}.$$

The preference lists of the men are given by

man	1^{st}	2^{nd}	3^{rd}	4^{th}
m_1	w_1	w_2	w_3	w_4
m_2	w_2	w_1	w_3	w_4
m_3	w_3	w_4	w_1	w_2
m_4	w_4	w_3	w_1	w_2

and the preference lists of the women are

woman	1^{st}	2^{nd}	3^{rd}	4^{th}
w_1	m_2	m_1	m_3	m_4
w_2	m_1	m_2	m_3	m_4
w_3	m_4	m_3	m_1	m_2
w_4	m_3	m_4	m_1	m_2

Now consider the matching

$$M = \{\{m_1, w_2\}, \{m_2, w_1\}, \{m_3, w_4\}, \{m_4, w_3\}\}.$$

We can see that M is stable. Indeed, each woman is matched to her first choice:

w_1 is matched with m_2, w_2 is matched with m_1, w_3 is matched with m_4, w_4 is matched with m_3.

Hence, no woman prefers any other man to her assigned partner. Therefore, no blocking pair can exist, since a blocking pair would require both to prefer each other over their current partners. Consequently, M is a stable matching. Moreover, this stable matching is not unique, since

$$M' = \{\{m_1, w_1\}, \{m_2, w_2\}, \{m_3, w_3\}, \{m_4, w_4\}\}$$

is also stable.

Example 1.3.2. *Consider an instance of four men*

$$\{m_1, m_2, m_3, m_4\}$$

and four women

$$\{w_1, w_2, w_3, w_4\}.$$

The preference lists of the men are given by

man	1^{st}	2^{nd}	3^{rd}	4^{th}
m_1	w_1	w_2	w_3	w_4
m_2	w_2	w_3	w_1	w_4
m_3	w_2	w_1	w_4	w_3
m_4	w_3	w_4	w_1	w_2

and the preference lists of the women are

woman	1^{st}	2^{nd}	3^{rd}	4^{th}
w_1	m_3	m_1	m_2	m_4
w_2	m_1	m_2	m_3	m_4
w_3	m_2	m_4	m_1	m_3
w_4	m_4	m_3	m_1	m_2

Now consider the matching

$$M = \{\{m_1, w_1\}, \{m_2, w_2\}, \{m_3, w_4\}, \{m_4, w_3\}\}.$$

We claim that M is not stable. Indeed, the pair $\{m_3, w_1\}$ is a blocking pair, since m_3 prefers w_1 to w_4, while w_1 prefers m_3 to m_1. Thus, both m_3 and w_1 prefer each other to their assigned partners under M. Hence, M admits a blocking pair and therefore it is not a stable matching.

Gale and Shapley [GS62] provided an algorithm that always outputs a stable marriage. The algorithm is quite simple. In each iteration of the algorithm, we try to match a man who is not matched to any woman by proposing to the woman he currently prefers most, and the woman accepts the best proposal she receives. If a man is matched with a woman and she later receives a better proposal, then she will leave her current partner and accept that proposal. The man will become unmatched again, and we will try to match him again in a later iteration, starting from the next woman in his preference list.

Algorithm 1.6 Gale–Shapley Algorithm

Require: Preference arrays *men*, *women* and their size n
Ensure: Array *manPartner* contains a stable matching

```
1: function StableMarriage(int n, int men[][], int women[][])
2:     initialize empty queue Q;
3:     for i ← 0 to n − 1 do
4:         ptr[i] ← 0;
5:         womanPartner[i] ← ∅;
6:         manPartner[i] ← ∅;
7:         Q.push(i);
8:     end for
9:     for i ← 0 to n − 1 do
10:        for j ← 0 to n − 1 do
11:            rank[i][women[i][j]] ← j;
12:        end for
13:    end for
14:    while not Q.isEmpty() do
15:        m ← Q.pop();
16:        while manPartner[m] = ∅ do
17:            w ← men[m][ptr[m]];
18:            ptr[m] + +;
19:            if womanPartner[w] = ∅ then
20:                womanPartner[w] ← m;
21:                manPartner[m] ← w;
22:            else
23:                if rank[w][m] < rank[w][womanPartner[w]] then
24:                    manPartner[m] ← w;
25:                    Q.push(womanPartner[w]);
26:                    manPartner[womanPartner[w]] ← ∅;
27:                    womanPartner[w] ← m;
28:                end if
29:            end if
30:        end while
31:    end while
32:    return manPartner;
33: end function
```

The input of the algorithm is the size n and the preference lists *women* and *men*. *women*[i] is the preference list of woman i and *men*[j] is the preference list of man j. Arrays *womanPartner* and *manPartner* represent the current matching. We will use two extra arrays that will help us see if a proposal can be made and whether it benefits us. Array *ptr* will store, for a man i, the most highly preferred woman that he has not proposed to yet. Using this array, we will be able to know the answer immediately. Array *rank* will help us determine, for a woman, whether she prefers the current matched man or the proposing one in $O(1)$. Specifically, for a woman i, we store at *rank*[i][j] the position of the man j in her preference list. This (lines 9-13), requires $O(n^2)$ preprocessing. In lines $14 - 31$ we perform the proposals.

See that if a woman receives a proposal, then she will either accept it or stay with her

current partner. This leads to the following lemma.

Lemma 1.3.3. *After a woman receives a match, she never becomes unmatched again.*

Note that it is not necessary to check whether $ptr[m]$ becomes greater than $n-1$, which would mean that we popped a man who is unmatched, but all the women are already matched.

Lemma 1.3.4. *Algorithm 1.6 always halts and produces a marriage.*

Proof. Assume that a man gets popped in line 15, which means that he is unmatched, but has already proposed to all women. Then every woman has rejected him for some other man and, by Lemma 1.3.3, all women are matched. This is a contradiction, because this would mean that a man appears in more than one match, so necessarily an unmatched woman exists. Consequently, every time we pop a man from the queue, there will always exist a woman who is unmatched. Since each man proposes to each woman at most once, there exist at most n^2 proposals, so the algorithm halts. Finally, when the algorithm halts, no man is unmatched, hence the output is a marriage. □

Now we are going to prove the output of the algorithm is stable.

Lemma 1.3.5. *Algorithm 1.6 produces a stable marriage.*

Proof. By Lemma 1.3.4, we know that it produces a marriage. Assume, to the contrary, that the output is not stable, which means that a blocking pair (m_i, w_j) exists. Because (m_i, w_j) does not belong to the matching, two cases are possible.

In the first case, the man m_i proposed to w_j and she rejected him (immediately or accepted him temporarily and later left him for a better proposal). This means that w_j prefers the man from the matching. A contradiction.

In the second case, the man m_i never proposed to w_j, which means that he matched with a woman who is higher on his preference list. Also a contradiction. □

As we perform $O(n^2)$ proposals and we can decide whether to accept a proposal or not in $O(1)$ time (using $O(n^2)$ preprocessing), the total time complexity is $O(n^2)$.

Theorem 1.3.6. *Algorithm 1.6 computes a stable marriage in $O(n^2)$ time.*

Definition 1.3.7. *Algorithm 1.6 operates as a men-proposing algorithm since a man first proposes to his most preferred woman, leaving the woman to accept the best proposal available to her.*

Theorem 1.3.8. *The men-proposing Algorithm 1.6 is man-optimal, as every man is matched to the best partner he could possibly obtain in any stable marriage.*

Proof. Suppose, for the sake of contradiction, that it is not man-optimal, and that the output of Algorithm 1.6 is a marriage M. This means that there exist some rejections in our algorithm such that the corresponding pairs are matched in another stable marriage M'. From those rejections, fix the first one. That is, a man m_i got rejected by woman w_j because w_j prefers m_z to m_i, and another stable marriage M' exists, where m_i is matched with w_j, i.e., $\{m_i, w_j\} \in M'$, and m_z is matched with w_k, i.e., $\{m_z, w_k\} \in M'$. If m_z prefers w_j to w_k, then $\{m_z, w_j\}$ is a blocking pair in M', which leads to a contradiction. If m_z prefers w_k to w_j, then it would be impossible for the first rejection to be that of m_i by w_j, because it would mean that w_j is already matched with m_z, implying that m_z was rejected by w_k in some previous iteration. This also leads to a contradiction. □

Although the output of the algorithm is man-optimal, it is the worst matching for women.

Theorem 1.3.9. *The men-proposing Algorithm 1.6 is female-pessimal, meaning every woman is matched with the least preferred valid partner she could have in any stable matching.*

Proof. Similar to the previous proof, assume to the contrary that woman w_j is matched with man m_i in the marriage output M of Algorithm 1.6 and that there exists another stable marriage $M^{'}$ such that w_j is matched with a man m_z whom she prefers less than m_i and m_i is matched with a woman w_k. Because M is man-optimal by Theorem 1.3.8, m_i prefers w_j to w_k. The pair $\{m_i, w_j\}$ is a blocking pair for $M^{'}$. This is a contradiction, since $M^{'}$ is stable. □

Further Variants of the Model. The Gale and Shapley algorithm focuses on a one-to-one matching model, which means that each element from one set can be matched to exactly one element from the other set. Also, the sizes of the sets and the preference lists are the same. These assumptions are actually rare. In real-life scenarios, we may have many-to-one or even many-to-many assignments, incomplete lists, and preferences with ties. Additionally, stability is just one of many criteria used to evaluate the quality of a matching. In a later chapter, we will study some of these extended scenarios.

Chapter 2

Polyhedral Approaches to Matching Problems

2.1 Preliminaries

There exist many excellent references on linear programming and polyhedral combinatorics. This section is based mainly on [MG07] and [Sch03].

Let C be any subset of $\mathbb{R}^n$. We say that C is *convex* if for every two points $u, v \in C$ and for every $\lambda \in [0, 1]$, $u + \lambda d$ belongs to C, where $d = v - u$. In other words, we say that C is convex if for every $u, v \in C$, it also contains the line segment between u and v. It is easy to see that the intersection of convex sets is convex.

Let $S \subset \mathbb{R}^n$ be a set. The *convex hull* of S, denoted by $conv(S)$, is the intersection of all the convex sets containing S. Expressed differently, it is the smallest convex set containing S. It is often more useful to describe the convex hull in terms of convex combinations.

Let $v_1, \ldots, v_k \in \mathbb{R}^n$ and $\lambda_1, \ldots, \lambda_k$ be non-negative reals such that $\lambda_1 + \ldots + \lambda_k = 1$. Then, $\lambda_1 v_1 + \ldots + \lambda_k v_k$ is called a *convex combination* of $v_1, \ldots, v_k$. Geometrically, convex combinations can be viewed as taking the line segments between the given points (by setting all coefficients to zero except those corresponding to two points) and then generating all points contained in the resulting shape as more coefficients are allowed to be non-zero. For two points, this generates the entire line segment connecting them, and for three *non-collinear* points, it generates the whole triangle (including its boundary). The set of all such points forms a convex set, called the convex hull of the given set. Therefore, $\text{conv}(S) = \{\sum_{i=1}^{k} \lambda_i v_i : k \in \mathbb{N}, v_i \in S, \lambda_i \geqslant 0, \sum_{i=1}^{k} \lambda_i = 1\}$, which is equivalent to the previous definition.

A set $S \subseteq \mathbb{R}^n$ is an *affine subspace* of $\mathbb{R}^n$ if $S = p + V = \{p + v : v \in V\}$ for some $p \in \mathbb{R}^n$ and some linear subspace V of $\mathbb{R}^n$. In other words, it is a linear subspace that has been shifted by a fixed vector. If $p \in V$, then the subspace is simply a linear subspace.

Recall the definition of a convex combination, but let $\lambda_i \in \mathbb{R}$ instead of $\lambda_i \in [0, 1]$. For two points, the result is the line passing through these two points. This line is formed by the affine combinations of these points (and it does not need to pass through the origin). In general, given $v_1, \ldots, v_k \in \mathbb{R}^n$ and $\lambda_1, \ldots, \lambda_k \in \mathbb{R}$ such that $\lambda_1 + \ldots + \lambda_k = 1$, the sum $\lambda_1 v_1 + \ldots + \lambda_k v_k$ is called an *affine combination* of $v_1, \ldots, v_k$.

The dimension of an affine subspace S, denoted by $\dim(S)$, is defined to be $\dim(V)$. Using affine combinations, we can generate a line, a plane, or a *hyperplane*, which is an affine subspace of dimension $n - 1$. Similarly to the definition of a convex hull, given a set $S \subseteq \mathbb{R}^n$, the *affine hull* of S ($\text{aff}(S)$) is the intersection of all affine subspaces containing S, or equivalently, the set of all possible affine combinations of a finite number of elements of

S.

We are also interested in knowing which points contribute to "extending" the affine hull (i.e., increasing its dimension). For example, in $\mathbb{R}^2$, the points $(1,0)$, $(0,1)$, and $(1,1)$ are affinely independent because the first two define a line, and by adding $(1,1)$, we extend the affine hull to all of $\mathbb{R}^2$. We say that the points $v_1, \ldots, v_k \in \mathbb{R}^n$ are *affinely independent* if none of them can be written as an affine combination of the remaining points. It can be proved that $v_1, \ldots, v_k \in \mathbb{R}^n$ are affinely independent if and only if the vectors $v_2 - v_1, \ldots, v_k - v_1$ are linearly independent.

A hyperplane in $\mathbb{R}^n$ describes all the solutions $x \in \mathbb{R}^n$ of a single linear equation of the form $a_1x_1 + a_2x_2 + \ldots + a_nx_n = b$ where at least one a_i is not zero and $b \in \mathbb{R}$. We can also express it using two inequalities $a_1x_1 + a_2x_2 + \ldots + a_nx_n \leqslant b$ and $a_1x_1 + a_2x_2 + \ldots + a_nx_n \geqslant b$. These inequalities are called *half-spaces* (and more specifically *closed* half-spaces because they contain their boundary).

A *polyhedron* $P \subseteq \mathbb{R}^n$ is the intersection of a finite number of halfspaces. This means that we could write $P = \{x \in \mathbb{R}^n : Ax \leqslant b\}$ where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. It is easy to see that polyhedra are convex because halfspaces are convex. We call z an *extreme point* (or *vertex*) of P if $z \in P$ and there do not exist distinct $x, y \in P$ and $\lambda \in (0,1)$ such that $z = (1-\lambda)x + \lambda y$ (which means that z is not an internal point of any segment from x to y such that $x, y \in P$). The *dimension* of a polyhedron P, denoted by $\dim(P)$, is the minimum dimension of an affine subspace containing P.

Let $P = \{x \in \mathbb{R}^n : Ax \leqslant b\}$ and denote a_i^T to be the i-th row of A. For a point $x^* \in P$, we say that the inequality $a_i^T x \leqslant b_i$ is *active* at x^* if $a_i^T x^* = b_i$. Let $A^= x \leqslant b^=$ be the subsystem of $Ax \leqslant b$ active at x^*. A point $x^* \in P$ is an extreme point of P if and only if the active constraint vectors at x^* contain n linearly independent rows, i.e., $\operatorname{rank}(A^=) = n$. In general, if a polyhedron is described by both equalities and inequalities, $P = \{x \in \mathbb{R}^n : Dx = d,\ Ax \leqslant b\}$, then $x^* \in P$ is an extreme point of P if and only if $\operatorname{rank} \begin{bmatrix} D \\ A^= \end{bmatrix} = n$.

An inequality $a^T x \leqslant b$ is called *valid* for P if it is satisfied by every point of P, that is, if $a^T x \leqslant b$ for all $x \in P$. The set $F = \{x \in P : a^T x = b\}$ is called the *face* of P induced by this valid inequality. A face F of P is called a *facet* if $\dim(F) = \dim(P) - 1$. In this case, we say that the inequality $a^T x \leqslant b$ is *facet-defining* for P. Equivalently, if $d = \dim(P)$ and the inequality is not satisfied with equality by every point of P, then it is facet-defining if and only if its equality set contains d affinely independent points of P.

A polyhedron $P \subseteq \mathbb{R}^n$ is called a *polytope* if it is bounded (in other words there exists $M > 0$ such that $\|x\| < M$, $\forall x \in P$, where $\|\|$ is the euclidean norm). A polytope has a finite number of extreme points and every other point of that polytope can be expressed as a convex combination of its extreme points. In fact, if C is the set of the extreme points of P, we can describe it with the convex hull of C, which means that $P = \operatorname{conv}(C)$. Hence, there are two representations of a polytope P. The first one is by defining the linear inequalities $Ax \leqslant b$, which is called the *H-description* (or *linear description*) and the second one by its vertices, which is called the *V-description*.

A polyhedron $P \subseteq \mathbb{R}^n$ with at least one vertex is called *integral* if all of its vertices are integral. In particular, a polytope is integral if and only if $P = \operatorname{conv}(P \cap \mathbb{Z}^n)$.

Let $c \in \mathbb{R}^n$. In linear programming (LP) we are interested to maximize a function $c^T x$ (which is often called an *objective function*) over a polyhedron. This is a linear optimization problem, which we call it a *primal problem*. The *constraints* of that problem are the linear description of it. In this thesis, we will mostly use maximization problems with nonnegative

variables. Unless stated otherwise, the variables of a linear program are assumed to be real-valued. Hence, the primal problem can be expressed as

$$\begin{aligned} \max\ & c^T x \\ \text{s.t. } & Ax \leqslant b \\ & x \geqslant 0 \end{aligned} \tag{2.1}$$

Any vector $x \in \mathbb{R}^n$ satisfying all constraints is called a *feasible solution*. The set of all feasible solutions is called the *feasible region* and in the case of a linear program, this region is a polyhedron. Any feasible solution that achieves the maximum possible value of the objective function is called an *optimal solution*. There can be multiple optimal solutions, or none at all. If the objective function can attain arbitrarily large values, we say that the LP is *unbounded*. Such a problem can be solved using known algorithms that are polynomial in the input size, for example, the ellipsoid method.

We also recall the basic notion of linear programming duality. Consider the primal linear program of (2.1) where $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$ and $c \in \mathbb{R}^n$. The corresponding *dual problem* is

$$\begin{aligned} \min\quad & b^T y \\ \text{s.t.}\quad & A^T y \geqslant c, \\ & y \geqslant 0. \end{aligned}$$

The variables x are called the *primal variables*, while the variables y are called the *dual variables*. A vector satisfying the constraints of the primal problem is called a *primal feasible solution*, and a vector satisfying the constraints of the dual problem is called a *dual feasible solution*.

The weak duality theorem states that, for every primal feasible solution x and every dual feasible solution y, we have

$$c^T x \leqslant b^T y.$$

Therefore, the value of any dual feasible solution gives an upper bound on the value of any primal feasible solution.

The strong duality theorem states that, if the primal problem has an optimal solution and its optimal value is finite, then the dual problem also has an optimal solution and the two optimal values are equal.

We recall two standard facts relating linear programs and vertices of polyhedra.

Proposition 2.1.1. *Let $P \subseteq \mathbb{R}^n$ be a polyhedron, and let $v \in P$. If v is a vertex of P, then there exists a nonzero vector $c \in \mathbb{R}^n$ such that*

$$c^T v > c^T y$$

for every $y \in P\backslash\{v\}$.

Theorem 2.1.2. *(Fundamental theorem of linear programming) Let $P \subseteq \mathbb{R}^n$ be a polyhedron. If the linear program*

$$\max\{c^T x : x \in P\}$$

has an optimal solution and P has at least one vertex, then it has an optimal solution that is a vertex of P.

Consider a linear program $\max\{c^T x : Ax = b,\ x \geqslant 0\}$ (which is often called *standard form*), where $A \in \mathbb{R}^{m\times n}$ has rank m. A set $B \subseteq \{1,\dots,n\}$ with $|B| = m$ is called a *basis* if the submatrix A_B consisting of the columns of A indexed by B is nonsingular. The variables indexed by B are called *basic variables*, while the remaining variables are called *nonbasic variables*. Setting the nonbasic variables equal to 0 and solving $A_B x_B = b$ gives the associated *basic solution*, namely

$$x_B = A_B^{-1} b, \qquad x_N = 0.$$

If this solution also satisfies $x_B \geqslant 0$, then it is called a *basic feasible solution*.

For linear programs in standard form, the basic feasible solutions are exactly the extreme points of the feasible polyhedron. Therefore, if such a linear program has an optimal solution, then it has an optimal basic feasible solution.

If we consider the LP problem but restrict $x \in \mathbb{Z}^n_+$, it becomes an *integer program* (IP). Unlike LP, this problem is difficult, i.e., it is known to be NP-hard. However, if the feasible region of the LP relaxation is integral, then we can solve the LP instead of the IP and obtain an integer optimal solution. The problem is still difficult if the size of the linear description is exponential in n or m. One way to address this issue is by solving the *separation* problem, in which, for a given point $x \in \mathbb{R}^n_+$, we decide whether it violates any of the inequalities of $Ax \leqslant b$ and return one such inequality (we will use this technique in the last chapter).

A matrix A is *totally unimodular* if every square submatrix of A has determinant equal to 0, 1 or -1. We will prove the following lemma.

Lemma 2.1.3. *Let A be a totally unimodular matrix and let A' be the matrix obtained from A by appending a unit vector e_i as a new column. Then A' is totally unimodular.*

Proof. Let B be an arbitrary square submatrix of A'. If B does not contain the appended column e_i, then B is a square submatrix of A, and therefore $\det(B) \in \{-1, 0, 1\}$.

Suppose now that B contains the appended column. The restriction of e_i to the rows of B is either the zero vector or a unit vector. In the first case, B has a zero column, so $\det(B) = 0$. In the second case, expanding along this column gives

$$\det(B) = \pm \det(C),$$

where C is a square submatrix of A. Since A is totally unimodular, $\det(C) \in \{-1, 0, 1\}$, and hence $\det(B) \in \{-1, 0, 1\}$.

Thus every square submatrix of A' has determinant in $\{-1, 0, 1\}$, so A' is totally unimodular. □

An *LP relaxation* of an IP is obtained by removing the integrality constraints on the variables and allowing them to take arbitrary real values within the feasible region. We now show how total unimodularity can be used to solve IP via LP relaxation, following the approach of [MG07, p. 145].

Lemma 2.1.4. *Let $\max\{c^T x : Ax \leqslant b,\ x \geqslant 0\}$ be the LP relaxation of an IP, where A is totally unimodular and $b \in \mathbb{Z}^m$. Suppose that the LP has an optimal solution. Then it also admits an integral optimal solution.*

Proof. Introduce slack variables $s \in \mathbb{R}^m$ and $s \geqslant 0$, so that $Ax + s = b$. Then the LP can be written in standard form as

$$\max\left\{ \begin{bmatrix} c \\ 0 \end{bmatrix}^T \begin{bmatrix} x \\ s \end{bmatrix} : \begin{bmatrix} A & I_m \end{bmatrix} \begin{bmatrix} x \\ s \end{bmatrix} = b,\ x \geqslant 0,\ s \geqslant 0 \right\}.$$

Let

$$M = \begin{bmatrix} A & I_m \end{bmatrix}.$$

Since A is totally unimodular, and I_m is obtained by appending the unit vectors $e_1, \ldots, e_m$, the previous result implies that $M = [A\ I_m]$ is also totally unimodular.

By the fundamental theorem of linear programming, since the linear program has an optimal solution, it has an optimal basic feasible solution (we could get one using the simplex method). Let

$$z^* = \begin{bmatrix} x^* \\ s^* \end{bmatrix}$$

be such an optimal basic feasible solution. Then there exists a basis matrix M_B, consisting of linearly independent columns of M, such that

$$z_B^* = M_B^{-1} b,$$

and all nonbasic variables are equal to 0.

Now M_B is a nonsingular square submatrix of M. Since M is totally unimodular,

$$\det(M_B) \in \{-1, 0, 1\}.$$

But M_B is nonsingular, so

$$\det(M_B) = \pm 1.$$

Therefore,

$$M_B^{-1} = \frac{\operatorname{adj}(M_B)}{\det(M_B)}$$

has integer entries, because M_B has integer entries and $\det(M_B) = \pm 1$. Since $b \in \mathbb{Z}^m$, it follows that

$$z_B^* = M_B^{-1} b$$

is integral. The nonbasic variables are zero, hence they are also integral. Therefore,

$$z^* = \begin{bmatrix} x^* \\ s^* \end{bmatrix} \in \mathbb{Z}^{n+m}.$$

In particular,

$$x^* \in \mathbb{Z}^n.$$

Thus the original linear program has an integral optimal solution. □

Consequently, we can deduce the following corollary.

Corollary 2.1.5. *The feasible region of a linear program with totally unimodular constraint matrix and integral right-hand side forms an integral polyhedron.*

To see why, suppose for contradiction, that P has a non-integral vertex v. Since v is a vertex, there exists a vector c such that v is the unique optimal solution of $\max\{c^T x : x \in P\}$. Since the constraint matrix is totally unimodular and the right-hand side is integral, the above linear program admits an integral optimal solution x^*. However, the optimal solution is unique, and therefore $x^* = v$. This implies that v is integral, which contradicts our assumption. Hence every vertex of P is integral, and so P is integral.

2.2 The Bipartite Matching Polytope

We solved the maximum bipartite matching problem algorithmically in Section 1.2. In this section, we will solve the same problem in terms of linear programming. Let $G = (V_1 \cup V_2, E)$ be a bipartite graph with n vertices $v_1, \ldots, v_n$ and m edges $e_1, \ldots, e_m$ and let $\delta(v)$ denote the set of edges incident to v. The IP describing the problem is the following:

$$\begin{aligned} \max \quad & \sum_{e \in E} x_e \\ \text{s.t.} \quad & \sum_{e \in \delta(v)} x_e \leqslant 1, \quad \forall v \in V(G), \\ & x_e \geqslant 0, \quad \forall e \in E, \\ & x_e \in \mathbb{Z}, \quad \forall e \in E. \end{aligned} \tag{2.2}$$

or in compact form:

$$\begin{aligned} \max \quad & \sum_{j=1}^{m} x_j \\ \text{s.t.} \quad & Ax \leqslant \mathbf{1}, \\ & x \geqslant 0, \\ & x \in \mathbb{Z}^m. \end{aligned} \tag{2.3}$$

where $\mathbf{1}$ is a vector with n components, each equal to 1 and the matrix A is defined as:

$$A_{v,e} = \begin{cases} 1, & \text{if } v \text{ is incident to edge } e, \\ 0, & \text{otherwise.} \end{cases}$$

In fact, x's components can be 0 or 1 based on the constraint matrix, so we could simply write $x_j \in \{0, 1\}$. Consider now the LP relaxation of (2.3).

$$\begin{aligned} \max \quad & \sum_{j=1}^{m} x_j \\ \text{s.t.} \quad & Ax \leqslant \mathbf{1}, \\ & x \geqslant 0. \end{aligned} \tag{2.4}$$

Since the feasible region is non-empty and bounded, it is a polytope, guaranteeing that the LP relaxation has an optimal solution at a vertex. The polytope of (2.4) is integral. To prove it, it's sufficient (see Corollary 2.1.5) to show that A is totally unimodular, since the right-hand side vector $\mathbf{1}$ is integral.

Lemma 2.2.1. *Let $G = (V_1 \cup V_2, E)$ be a bipartite graph, and let A be its vertex-edge incidence matrix. Then A is totally unimodular.*

Proof. We prove that every square submatrix of A has determinant equal to 0, 1, or -1.

Let Q be an arbitrary square submatrix of A. We use induction on the size of Q. If Q is a 1×1 matrix, then its determinant is either 0 or 1, since all entries of A are 0 or 1.

Now suppose that Q has size $k \times k$, with $k \geqslant 2$, and assume that the claim holds for all smaller square submatrices of A.

Since A is the vertex-edge incidence matrix of G, every column of A contains exactly two entries equal to 1, one corresponding to each endpoint of the edge. Therefore, every column of Q contains at most two entries equal to 1.

If some column of Q contains no entry equal to 1, then this column is zero, and hence

$$\det(Q) = 0.$$

If some column of Q contains exactly one entry equal to 1, then we expand the determinant along this column. We get

$$\det(Q) = \pm \det(Q'),$$

where Q' is a square submatrix of A of size $(k-1) \times (k-1)$. By the induction hypothesis,

$$\det(Q') \in \{-1, 0, 1\}.$$

Therefore,

$$\det(Q) \in \{-1, 0, 1\}.$$

It remains to consider the case where every column of Q contains exactly two entries equal to 1. Since G is bipartite, every edge has one endpoint in V_1 and one endpoint in V_2. Hence, in every column of Q, one of the two 1's appears in a row corresponding to a vertex of V_1, and the other appears in a row corresponding to a vertex of V_2.

Therefore, the sum of the rows of Q corresponding to vertices in V_1 is equal to the sum of the rows of Q corresponding to vertices in V_2. Hence, the rows of Q are linearly dependent, and so

$$\det(Q) = 0.$$

In all cases, we have

$$\det(Q) \in \{-1, 0, 1\}.$$

□

Since the polytope of the LP relaxation contains all feasible solutions of the IP, the LP relaxation can only have an optimal value at least as large as that of the IP. However, the polytope of the LP relaxation is integral and since it has an optimal solution at a vertex, this optimal solution is also integral. This optimal solution is feasible for the original IP as well, since each component of the solution is either 0 or 1. Consequently, the LP relaxation and the IP have the same optimal value, so we can solve the LP instead of the IP to obtain a maximum matching.

Another famous problem in graph theory, is the *minimum vertex cover*. A *vertex cover* of a graph G is a set of nodes such that each edge of the graph has at least one endpoint in the set. The minimum vertex cover problem asks for a vertex cover of minimum cardinality. This problem is NP-hard in a general graph. However, if the graph is bipartite, we can solve it in polynomial time. In fact, the following theorem holds.

Theorem 2.2.2 (König's Theorem)**.** *Let $G = (V_1 \cup V_2, E)$ be a bipartite graph. Then the maximum cardinality of a matching in G is equal to the minimum cardinality of a vertex cover in G.*

Proof. Let A be the vertex-edge incidence matrix of G. Consider the LP relaxation (2.4) of the maximum matching problem. We have shown that, since A is totally unimodular and the right-hand side vector is integral, the optimal value of this relaxation is equal to the maximum cardinality of a matching in G.

The dual of this LP relaxation is

$$\begin{aligned} \min \quad & \mathbf{1}^T y \\ \text{s.t.} \quad & A^T y \geqslant \mathbf{1}, \\ & y \geqslant 0. \end{aligned}$$

This is exactly the LP relaxation of the minimum vertex cover problem. Indeed, the IP for minimum vertex cover is

$$\begin{aligned} \min \quad & \mathbf{1}^T y \\ \text{s.t.} \quad & A^T y \geqslant \mathbf{1}, \\ & y \geqslant 0, \\ & y \in \mathbb{Z}^n. \end{aligned}$$

Here $y_v = 1$ means that the vertex v is chosen in the vertex cover. The constraint $A^T y \geqslant \mathbf{1}$ says that for every edge, at least one of its endpoints must be chosen.

Since A is totally unimodular, it is easy to see that A^T is also totally unimodular. Therefore, again by the total unimodularity result, the LP relaxation of the minimum vertex cover problem has an integral optimal solution. Moreover, in an optimal integral solution we may assume that every component is either 0 or 1, since if $y_v > 1$, replacing y_v by 1 keeps all constraints feasible and does not increase the objective value (in fact, it is impossible to happen, since this would strictly decreases the objective value, contradicting optimality). Hence, the optimal value of the vertex cover LP relaxation is equal to the minimum cardinality of a vertex cover in G.

Finally, by strong duality, the optimal values of the two LP relaxations are equal. The first one is equal to the maximum cardinality of a matching in G, and the second one is equal to the minimum cardinality of a vertex cover in G. Therefore, these two quantities are equal. □

2.3 The Stable Matching Polytope

In Section 2.3, we studied the stable marriage problem from an algorithmic point of view. In particular, the Gale–Shapley algorithm computes a stable marriage in polynomial time [GS62]. In this section, we study the same problem from a polyhedral point of view.

We keep the same assumptions as before. There are n men and n women, all preference lists are complete and strict, and every feasible solution is a marriage. Let

$$U = \{m_1, m_2, \ldots, m_n\}$$

be the set of men and

$$V = \{w_1, w_2, \ldots, w_n\}$$

be the set of women. For a man m, we write $w' \succ_m w$ if m prefers woman w' to woman w. Similarly, for a woman w, we write $m' \succ_w m$ if w prefers man m' to man m.

For every pair $(m, w) \in U \times V$, define a variable

$$x_{mw} = \begin{cases} 1, & \text{if } m \text{ is matched with } w, \\ 0, & \text{otherwise.} \end{cases}$$

Then the stable marriage problem can be written as the following IP:

$$\begin{aligned}
\sum_{w \in V} x_{mw} &= 1, && \forall m \in U, \\
\sum_{m \in U} x_{mw} &= 1, && \forall w \in V, \\
x_{mw} + \sum_{\substack{w' \in V \\ w' \succ_m w}} x_{mw'} + \sum_{\substack{m' \in U \\ m' \succ_w m}} x_{m'w} &\geqslant 1, && \forall (m,w) \in U \times V, \\
x_{mw} &\in \{0,1\}, && \forall (m,w) \in U \times V.
\end{aligned} \tag{2.5}$$

The first two families of constraints say that each man is matched with exactly one woman and each woman is matched with exactly one man. Hence, they describe a marriage.

The third family of constraints expresses stability. Fix a pair (m, w). The inequality says that at least one of the following must happen:

$$m \text{ is matched with } w,$$

or

$$m \text{ is matched with a woman he prefers to } w,$$

or

$$w \text{ is matched with a man she prefers to } m.$$

If none of these happens, then m and w prefer each other to their assigned partners. Therefore, (m, w) is a blocking pair.

Lemma 2.3.1. *The feasible integer points of (2.5) are exactly the incidence vectors of stable marriages.*

Proof. Let x be a feasible integer point of (2.5). Since $x_{mw} \in \{0, 1\}$, the first two families of constraints define a marriage. Let N be this marriage.

We show that N is stable. Suppose for the sake of contradiction that there exists a blocking pair (m, w). Then m is not matched with w, so

$$x_{mw} = 0.$$

Also, since m prefers w to his assigned partner, m is not matched with any woman w' such that $w' \succ_m w$. Thus,

$$\sum_{\substack{w' \in V \\ w' \succ_m w}} x_{mw'} = 0.$$

Similarly, since w prefers m to her assigned partner, w is not matched with any man m' such that $m' \succ_w m$. Hence,

$$\sum_{\substack{m' \in U \\ m' \succ_w m}} x_{m'w} = 0.$$

Therefore, the stability constraint for the pair (m, w) is violated. This is a contradiction. Hence, N is stable.

Conversely, let N be a stable marriage and let χ^N be its incidence vector. The first two families of constraints are clearly satisfied. Now fix a pair (m, w). If m is matched with w, then the stability inequality is satisfied because $x_{mw} = 1$. Otherwise, since N is stable, the pair (m, w) is not blocking. Therefore, either m is matched with a woman he prefers to w, or w is matched with a man she prefers to m. Hence, the stability inequality is again satisfied. □

We now define the polytope associated with stable marriages.

Definition 2.3.2. *The stable matching polytope, also called the stable marriage polytope in this setting, is*

$$P_{SM} = \operatorname{conv}\left\{\chi^N \in \mathbb{R}^{U\times V} : N \text{ is a stable marriage}\right\}.$$

Thus, P_{SM} is the convex hull of all incidence vectors of stable marriages. It contains all stable marriages as integral points, but it may also contain fractional points. For example, if N_1 and N_2 are two different stable marriages, then $\frac{1}{2}\chi^{N_1} + \frac{1}{2}\chi^{N_2}$ is usually fractional and also belongs to P_{SM}.

Let Q_{SM} be the polyhedron obtained from (2.5) by replacing the constraints $x_{mw} \in \{0,1\}$ with $x_{mw} \geqslant 0$. Since the first two families of constraints imply $x_{mw} \leqslant 1$, no separate upper bound is needed.

The important question is whether this linear relaxation introduces fractional vertices. In fact, the relaxation is *exact*.

Theorem 2.3.3 (Vande Vate [Vat89]; Rothblum [Rot92])**.** *For every stable marriage instance with complete and strict preference lists,*

$$Q_{SM} = P_{SM}.$$

Equivalently, every extreme point of Q_{SM} is the incidence vector of a stable marriage.

The nontrivial part of the theorem is that the polyhedron Q_{SM} has no fractional extreme points, so Q_{SM} is *integral*. Therefore, every fractional feasible point of Q_{SM} can be written as a convex combination of incidence vectors of stable marriages, and is not a new vertex of the relaxation.

This gives a complete linear description of the stable matching polytope:

$$P_{SM} = \left\{ x \in \mathbb{R}^{U\times V}_{\geqslant 0} : \begin{array}{ll} \sum_{w\in V} x_{mw} = 1, & \forall m \in U, \\ \sum_{m\in U} x_{mw} = 1, & \forall w \in V, \\ x_{mw} + \sum_{\substack{w'\in V \\ w' \succ_m w}} x_{mw'} + \sum_{\substack{m'\in U \\ m' \succ_w m}} x_{m'w} \geqslant 1, & \forall (m,w) \in U \times V. \end{array} \right\}.$$

The formulation has n^2 variables, $2n$ assignment constraints, and n^2 stability constraints, apart from non-negativity. Hence, it is a compact linear formulation.

Consequences. The Gale–Shapley algorithm is enough if we only want to find one stable marriage. The polyhedral formulation is useful when we want to optimize a linear objective over all stable marriages. For example, if each pair (m,w) has weight c_{mw}, we may solve

$$\begin{aligned} \max \quad & \sum_{m\in U}\sum_{w\in V} c_{mw}x_{mw} \\ \text{s.t.} \quad & x \in Q_{SM}. \end{aligned} \tag{2.6}$$

Because every extreme point of Q_{SM} is integral, there exists an optimal extreme point of (2.6) that is the incidence vector of a stable marriage. Therefore, the optimal stable marriage problem with a linear objective can be solved as an LP.

This section gives only a brief polyhedral view of the stable marriage problem. We introduced the natural integer formulation, defined the stable matching polytope, and stated the main result that its linear relaxation is integral. Together with the maximum bipartite matching formulation studied in the previous section, this gives another example where the natural linear relaxation is integral. However, this is not the case for many other matching problems. In the next chapter, we will study one such case.

Chapter 3

Pareto Matchings in one-to-one Models

3.1 Model and Definitions

In Sections 2.3 and 3.3, we studied the stable marriage problem, where there are n men and n women, and each participant has a preference list containing all members of the opposite side. Therefore, every stable matching matches all participants.

In this chapter, we study a different setting, where the two sides need not have the same size, and only one side has preferences over the other side. For these reasons, it is natural to formulate this matching problem in the setting of a housing market.

The *house allocation problem* has been studied from an algorithmic point of view in [ACMM04]. An integer programming formulation for the same setting was later proposed by Eirinakis, Magos, and Mourtos [EMM15]. In this chapter, we follow the notation and the basic definitions used in these works.

House allocation is a one-sided matching problem. We are given a finite set

$$A = \{a_1, \ldots, a_r\}$$

of applicants and a finite set

$$H = \{h_1, \ldots, h_s\}$$

of houses. The houses are indivisible objects, and no monetary transfers are allowed. In contrast with two-sided matching models, only the applicants have preferences.

Each applicant $a \in A$ finds acceptable only some houses in H. We denote by $P(a)$ the preference list of applicant a. The list $P(a)$ is strict, so no ties are allowed. If $h, h' \in P(a)$, then

$$h >_a h'$$

means that applicant a prefers house h to house h'.

The acceptable pairs define a bipartite graph

$$G = (A, H, E),$$

where

$$E = \{(a, h) \in A \times H : h \in P(a)\}.$$

Thus, an edge (a, h) means that house h is acceptable to applicant a.

For each house $h \in H$, let

$$Q(h) = \{a \in A : h \in P(a)\}.$$

Therefore, $Q(h)$ is the set of applicants whose preference list contains house h.

A matching is a set $M \subseteq E$ such that each applicant is assigned to at most one house and each house is assigned to at most one applicant. If $(a, h) \in M$, then we write

$$M(a) = h \quad \text{and} \quad M(h) = a.$$

If an applicant or a house is not assigned in M, then we write

$$M(z) = \varnothing, \quad z \in A \cup H.$$

Thus, in this model, every feasible allocation is *one-to-one*: no applicant receives more than one house and no house is given to more than one applicant.

A matching M' is a *Pareto improvement* over a matching M if every applicant is at least as well off in M' as in M, and at least one applicant is strictly better off. Here, being unmatched is considered worse than being assigned any acceptable house. A matching M is Pareto optimal if no Pareto improvement over M exists.

We next recall three properties of matchings that will be useful for describing Pareto optimality.

A matching M is *maximal* if there is no acceptable pair $(a, h) \in E$ such that

$$M(a) = M(h) = \varnothing.$$

In other words, no additional acceptable pair can be added to M without violating the matching constraints.

A matching M is *trade-in free* if there is no applicant $a \in A$ and no house $h \in P(a)$ such that

$$M(a) \neq \varnothing, \qquad M(h) = \varnothing, \qquad \text{and} \qquad h >_a M(a).$$

Thus, no matched applicant can improve the current assignment by replacing the assigned house with an unmatched house.

A matching M is *coalition free* if it does not admit a coalition. A coalition is a sequence of matched applicants

$$C = \langle a_{i_0}, a_{i_1}, \ldots, a_{i_{k-1}} \rangle, \quad k \geqslant 2,$$

such that

$$M(a_{i_{c+1}}) >_{a_{i_c}} M(a_{i_c}), \quad c = 0, \ldots, k-1,$$

where the indices are taken modulo k. If such a sequence exists, then the applicants in C can cyclically exchange their assigned houses, and every applicant in the sequence becomes strictly better off.

The following characterization connects these three properties with Pareto optimality.

Proposition 3.1.1 ([ACMM04, Chapter 2, p. 6])**.** *A matching M is Pareto optimal if and only if M is maximal, trade-in free, and coalition free.*

This characterization is the basis of the integer programming formulation that will be presented in the next section.

For this formulation, it is useful to define coalitions independently of a specific matching. A plausible coalition is a set of applicant-house pairs

$$C = \{(a_{i_c}, h_{i_c}) : c = 0, \dots, k-1\}, \quad k \geqslant 2,$$

such that, for every $c = 0, \dots, k-1$,

$$h_{i_{c+1}} >_{a_{i_c}} h_{i_c}, \qquad \{h_{i_c}, h_{i_{c+1}}\} \subseteq P(a_{i_c}),$$

where again the indices are taken modulo k. If all pairs of such a set appear simultaneously in a matching, then the corresponding applicants form a coalition. We denote by $\mathcal{C}$ the family of all plausible coalitions.

Finally, for every acceptable pair $(a, h) \in E$, we define a binary variable x_{ah}. For a matching M, its *incidence vector* is the vector $x^M \in \{0,1\}^E$, given by

$$x^M_{ah} = \begin{cases} 1, & \text{if } (a,h) \in M, \\ 0, & \text{otherwise.} \end{cases}$$

These incidence vectors will later be used to define the Pareto matching polytope.

3.2 An Integer Programming Formulation

We now present an integer programming formulation for Pareto optimal matchings, following Eirinakis, Magos, and Mourtos [EMM15]. The formulation uses the characterization stated in the previous section: a matching is Pareto optimal if and only if it is maximal, trade-in free, and coalition free.

For every acceptable pair (a, h), where $a \in A$ and $h \in P(a)$, we define a binary variable x_{ah}. The meaning of this variable is

$$x_{ah} = \begin{cases} 1, & \text{if applicant } a \text{ is assigned to house } h, \\ 0, & \text{otherwise.} \end{cases}$$

Thus, a vector x represents a set of applicant-house pairs, and the constraints below force this set to be a Pareto optimal matching.

First, we need the usual matching constraints. Each applicant can be assigned to at most one house, and each house can be assigned to at most one applicant. Hence, we require

$$\sum_{h \in P(a)} x_{ah} \leqslant 1, \qquad \forall a \in A,$$

and

$$\sum_{a \in Q(h)} x_{ah} \leqslant 1, \qquad \forall h \in H.$$

The first family of constraints controls the applicants, while the second family controls the houses.

Next, we impose maximality. Fix an acceptable pair (a, h). If applicant a is unmatched and house h is also unmatched, then the pair (a, h) could be added to the matching. This is exactly what maximality forbids. Therefore, for every $a \in A$ and every $h \in P(a)$, we require

$$\sum_{h' \in P(a)} x_{ah'} + \sum_{a' \in Q(h) \setminus \{a\}} x_{a'h} \geqslant 1.$$

The first sum says whether applicant a is matched to some house. The second sum says whether house h is matched to some applicant different from a. Hence, the constraint prevents the case in which a and h are both unmatched.

We also need to exclude trade-ins. Fix an applicant a and a house $h \in P(a)$. If house h is unmatched, then applicant a must not be assigned to a house that is less preferred than h. Otherwise, applicant a could drop the current assignment and move to the free house h. This is expressed by

$$\sum_{a' \in Q(h)} x_{a'h} - \sum_{\substack{h' \in P(a) \\ h >_a h'}} x_{ah'} \geqslant 0, \qquad \forall a \in A,\ \forall h \in P(a).$$

Indeed, if house h is unmatched, then the first sum is equal to 0. The constraint then forces all variables $x_{ah'}$ with $h >_a h'$ to be equal to 0. Thus, applicant a cannot be assigned to a worse house while h is free.

Finally, we need to exclude coalitions. Let $C \in \mathcal{C}$ be a plausible coalition. If all pairs in C appeared simultaneously in a matching, then the corresponding applicants would be able to cyclically exchange houses and all become strictly better off. Therefore, at least one pair of every plausible coalition must be absent from the matching. This gives the constraints

$$\sum_{(a,h) \in C} x_{ah} \leqslant |C| - 1, \qquad \forall C \in \mathcal{C}.$$

Putting all constraints together, we obtain the following integer programming formulation:

$$\begin{aligned}
\sum_{h \in P(a)} x_{ah} &\leqslant 1, && \forall a \in A, \\
\sum_{a \in Q(h)} x_{ah} &\leqslant 1, && \forall h \in H, \\
\sum_{h' \in P(a)} x_{ah'} + \sum_{a' \in Q(h) \setminus \{a\}} x_{a'h} &\geqslant 1, && \forall a \in A,\ \forall h \in P(a), \\
\sum_{a' \in Q(h)} x_{a'h} - \sum_{\substack{h' \in P(a) \\ h >_a h'}} x_{ah'} &\geqslant 0, && \forall a \in A,\ \forall h \in P(a), \\
\sum_{(a,h) \in C} x_{ah} &\leqslant |C| - 1, && \forall C \in \mathcal{C}, \\
x_{ah} &\in \{0,1\}, && \forall a \in A,\ \forall h \in P(a).
\end{aligned} \tag{3.1}$$

The first two families of constraints ensure that the selected pairs form a matching. The third family enforces maximality, the fourth family enforces the trade-in free property, and the fifth family excludes coalitions. Therefore, by the characterization of Pareto optimal matchings, the feasible integer points of (3.1) are exactly the incidence vectors of Pareto optimal matchings. We refer to these as POM vectors.

The formulation is written as a feasibility problem, since our goal is to describe the set of Pareto optimal matchings rather than optimize a particular objective function.

3.3 Linear Relaxation and the Separation Problem

The next step is to study the LP relaxation of (3.1) as we similarly did for other known problems in chapter 3. We saw that those problems had an integral feasible region, which

means we could obtain an optimal value by solving their LP relaxation, and, for this reason, get our answer in polynomial time. Unfortunately, this is not the case for the house allocation problem.

Consider the LP relaxation of (3.1) by removing the constraints $x_{ah} \in \{0,1\}, \forall a \in A,\ \forall h \in P(a)$ and adding $x_{ah} \geqslant 0, \forall a \in A,\ \forall h \in P(a)$. Because of the matching constraints, the feasible region is bounded. To prove that the polytope is not integral, it's sufficient to find a fractional vertex. To do this, we will actually do a brute force using the Proposition 2.1.1. More specifically, we will generate random coefficients to construct an objective function and then solving the resulting LP using the simplex method (which *guarantees* that we will get a vertex). We will repeat these steps until we find a fractional vertex.

Of course, the above method is really slow, since the polytope can be very large, with millions of integral vertices but only a few fractional ones. Nevertheless, we got to find one with the following input.

applicant	pref1	pref2	pref3	pref4
1	1	2	4	$\varnothing$
2	1	3	2	$\varnothing$
3	3	5	1	4
4	3	2	5	$\varnothing$
5	2	1	$\varnothing$	$\varnothing$

Table 3.1: Preference lists of the applicants

In the above input we have the preference lists of 5 applicants. There are 5 houses in total. Each row i is a preference list of the applicant i. With pref1 we mean the first preference of an applicant, pref2 the second one, etc. For example the applicant 4 prefers the house 3, than the house 2. We need 15 binary variables for this input, which means that each incidence vector will have 15 components.

To solve the linear relaxation we used the GLPK solver in SageMath [Sag26]. The variables were declared as continuous and nonnegative. The coefficients were:

$$\begin{aligned} \max \quad & -12x_{11} - 18x_{12} + 14x_{14} - 9x_{21} - 16x_{23} + 17x_{22} \\ & - 6x_{33} + 7x_{35} + 15x_{31} + 8x_{34} + 7x_{43} + 11x_{42} \\ & + 19x_{45} + 12x_{52} + 10x_{51}. \end{aligned}$$

Solving the linear relaxation with this objective function gives the following fractional optimal solution:

$$\begin{aligned} x_{11} &= 0, \quad x_{12} = 0, \quad x_{14} = 1, \\ x_{21} &= 0, \quad x_{23} = 0, \quad x_{22} = \frac{2}{3}, \\ x_{33} &= \frac{1}{3}, \quad x_{35} = 0, \quad x_{31} = \frac{2}{3}, \\ x_{34} &= 0, \quad x_{43} = \frac{1}{3}, \quad x_{42} = 0, \\ x_{45} &= \frac{2}{3}, \quad x_{52} = \frac{1}{3}, \quad x_{51} = \frac{1}{3}. \end{aligned} \tag{3.2}$$

which is a fractional vertex.

Proposition 3.3.1. *The linear relaxation of* (3.1) *is not integral in general.*

The code used for this experiment is given in Listing B.1. The listing contains the final version of the experiment, where some additional constraints have already been included. Since these constraints have not yet been discussed, the output does not coincide with the above solution obtained from the basic relaxation. We explain why in the final section.

Separation. Notice that the family of coalition constraints may be large. In fact, the size of $\mathcal{C}$ can be factorial (or even larger). Consider an instance with five applicants and twenty-five houses. Let the houses be divided into five blocks

$$H_1 = \{1, 2, 3, 4, 5\}, \quad H_2 = \{6, 7, 8, 9, 10\}, \quad H_3 = \{11, 12, 13, 14, 15\},$$

$$H_4 = \{16, 17, 18, 19, 20\}, \quad H_5 = \{21, 22, 23, 24, 25\}.$$

Denote the j-th house of block H_i by $h_{i,j}$. Assume that applicant a_i ranks all houses of H_{i+1} above all houses of H_i, where the index is taken cyclically ($H_6 = H_1$). Then, for every permutation π of the set $\{1, 2, 3, 4, 5\}$, the set

$$C_\pi = \{(a_1, h_{1,\pi(1)}), (a_2, h_{2,\pi(2)}), (a_3, h_{3,\pi(3)}), (a_4, h_{4,\pi(4)}), (a_5, h_{5,\pi(5)})\}$$

forms a plausible coalition. Indeed, applicant a_1 prefers $h_{2,\pi(2)}$ to $h_{1,\pi(1)}$, applicant a_2 prefers $h_{3,\pi(3)}$ to $h_{2,\pi(2)}$, and so on, while applicant a_5 prefers $h_{1,\pi(1)}$ to $h_{5,\pi(5)}$. Therefore, each such permutation defines a distinct plausible coalition. Since there are 5! permutations of $\{1, 2, 3, 4, 5\}$, this construction gives at least $5! = 120$ distinct plausible coalitions. More generally, the same construction with n applicants and n houses in each block gives at least $n!$ plausible coalitions. This shows why generating all these constraints upfront is impractical, making a separation procedure essential.

The envy graph used in [ACMM04, Chapter 2, p. 6] is defined with respect to a fixed matching M. In that graph, the vertices are the applicants, and an edge from a_i to a_j means that a_i would prefer the house assigned to a_j in M. Thus, for a matching M, coalitions can be detected by looking for directed cycles in this graph.

In our separation procedure, we use a slightly different auxiliary graph, since the current solution of the LP relaxation need not correspond to a matching. The vertices of this graph are the assignment pairs

$$V = \{(a, h) : h \in P(a)\}.$$

For two vertices (a, h) and (a', h'), with $a \neq a'$, we add a directed edge

$$(a, h) \longrightarrow (a', h')$$

whenever applicant a strictly prefers h' to h. Therefore, a directed cycle in this graph represents a possible cyclic improvement among assignment pairs, and hence corresponds to a plausible coalition. A small example of a coalition is illustrated in the following figure.

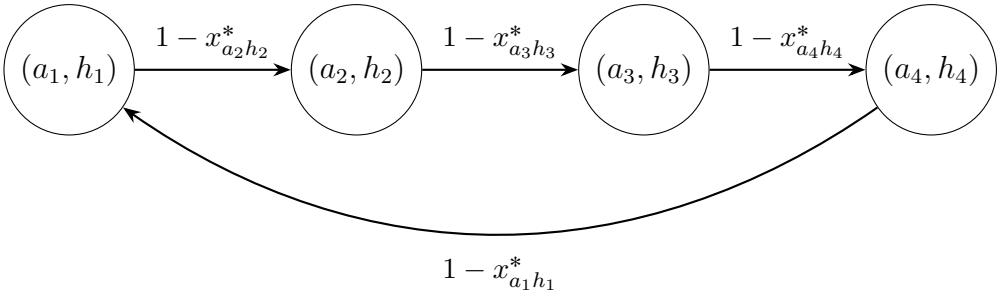

Figure 3.1: A directed cycle in the auxiliary graph used for separation. Each vertex is an assignment pair (a_i, h_i). The edge from (a_i, h_i) to (a_j, h_j) has weight $1 - x^*_{a_jh_j}$.

The directed cycle in the figure corresponds to the preference relations

$$h_2 \succ_{a_1} h_1, \qquad h_3 \succ_{a_2} h_2, \qquad h_4 \succ_{a_3} h_3, \qquad h_1 \succ_{a_4} h_4.$$

Let x^* be the current solution of the LP relaxation. To separate the coalition-free constraints, we assign to each vertex (a, h) the weight

$$w(a, h) = 1 - x^*_{ah}.$$

For a directed cycle C, the corresponding coalition inequality is

$$\sum_{(a,h)\in C} x_{ah} \leqslant |C| - 1.$$

At the point x^*, this inequality is violated if and only if

$$\sum_{(a,h)\in C} x^*_{ah} > |C| - 1,$$

or equivalently,

$$\sum_{(a,h)\in C} (1 - x^*_{ah}) < 1. \tag{3.3}$$

Hence, the separation problem reduces to finding a minimum-weight directed cycle in the auxiliary graph. If the weight of such a cycle is less than 1, then the corresponding coalition constraint is violated and is added to the relaxation. Otherwise, no coalition-free constraint is violated by the current LP solution.

The separation procedure is implemented as a cutting-plane procedure. We first solve the LP relaxation without adding any coalition-free constraints. Let x^* be the solution obtained from this relaxation.

Using x^*, we then construct the auxiliary directed graph described above. The unweighted structure of this graph depends only on the preference lists. However, in each iteration, the vertex weights depend on the current LP solution. More precisely, each vertex (a, h) receives weight

$$w(a, h) = 1 - x^*_{ah}.$$

Since we use Floyd–Warshall and this algorithm applies only to edge-weighted graphs, we convert these vertex weights into edge weights. Assume a directed edge $((a, h), (a', h'))$ and let $u = (a, h)$ and $v = (a', h')$. We then set the edge weight (w^*) equal to the weight of the head vertex:

$$w^*(u, v) = w(v).$$

With this convention, the weight of a directed cycle is exactly the sum of the weights of its vertices.

We then run the Floyd–Warshall algorithm on this weighted directed graph and obtain the all-pairs shortest-path distance matrix D. We do not inspect the diagonal entries $D_{u,u}$, since they correspond to the empty path and are equal to zero. Instead, for every directed edge (u, v), we check whether there is a path from v back to u. If such a path exists, then the edge (u, v) together with a shortest path from v to u forms a directed cycle. Its weight is

$$w^*(u, v) + D_{v,u}.$$

Therefore, by checking the quantity $w^*(u, v) + D_{v,u}$ over all directed edges (u, v), we find a minimum-weight directed cycle in the auxiliary graph.

If the minimum cycle C has weight strictly less than 1, then the coalition-free constraint

$$\sum_{(a,h)\in C} x_{ah} \leqslant |C| - 1$$

is violated by the current solution x^*. We add this inequality to the LP relaxation and solve the relaxation again.

The process is repeated until the separation step finds no additional violated coalition-free constraint. At that point, the current LP solution satisfies both the coalition constraints that have already been added explicitly and all remaining coalition constraints that have not been generated. Hence, instead of generating all constraints of $\mathcal{C}$ in advance, we add only those that are actually violated during the solution process.

Lemma 3.3.2. *The separation problem for the coalition-free constraints can be solved in polynomial time. More precisely, let $n = |A|$ be the number of applicants, let*

$$L = \max_{a\in A} |P(a)|$$

be the maximum length of a preference list, and let

$$r = \sum_{a\in A} |P(a)|$$

be the number of acceptable pairs. Then one separation call can be performed in $O(r^3)$ time. In particular, since $r \leqslant nL$, this is also $O(n^3L^3)$.

Proof. For each applicant $a \in A$, let $\ell_a = |P(a)|$ be the length of the preference list of a. Then

$$r = \sum_{a\in A} \ell_a$$

is the number of acceptable pairs, and therefore also the number of variables x_{ah}.

The auxiliary graph has one vertex for each acceptable pair (a, h). Hence,

$$|V_G| = r \leqslant nL.$$

We now estimate the number of edges. Fix a vertex (a, h), and suppose that h appears in position t of the preference list $P(a)$. Then applicant a prefers exactly $t - 1$ houses to h. For each such house h', there can be edges to vertices of the form (a', h'), with $a' \neq a$. In

the worst case, there are at most $n-1$ such vertices. Therefore, the outdegree of (a,h) is at most

$$(t-1)(n-1).$$

Summing over all vertices, we obtain

$$|E_G| \leqslant (n-1)\sum_{a\in A}\sum_{t=1}^{\ell_a}(t-1).$$

Since

$$\sum_{t=1}^{\ell_a}(t-1) = \frac{\ell_a(\ell_a-1)}{2},$$

we get

$$|E_G| \leqslant (n-1)\sum_{a\in A}\frac{\ell_a(\ell_a-1)}{2}.$$

In particular, since $\ell_a \leqslant L$ for every applicant a, we have

$$|E_G| \leqslant (n-1)n\frac{L(L-1)}{2} = O(n^2L^2).$$

Equivalently, since $r \leqslant nL$, we can write

$$|E_G| = O(r^2).$$

After the graph has been constructed, one separation call consists of the following steps. First, we assign weights to the vertices, which takes $O(r)$ time. Then we convert these vertex weights into edge weights, which takes $O(|E_G|)$ time. We then run the Floyd–Warshall algorithm on a graph with r vertices, which takes

$$O(r^3)$$

time. Finally, we inspect all directed edges (u,v) and check the value

$$w^*(u,v) + D_{v,u}.$$

This takes $O(|E_G|)$ time.

Hence, the total running time of one separation call is

$$O(r^3 + |E_G|) = O(r^3).$$

Since $r \leqslant nL$, this is also

$$O((nL)^3) = O(n^3L^3).$$

In the case of complete preference lists with n houses, we have $L = n$ and therefore $r \leqslant n^2$. In that case, the worst-case running time becomes

$$O(n^6).$$

□

3.4 The Pareto Matching Polytope

In this section, we study the polytope obtained from the POM vectors. The goal is not to give a general closed-form description of this polytope (which we think it's a hard task), but rather to examine it computationally for small instances. This allows us to compare the true convex hull of Pareto optimal matchings with the linear relaxation studied in the previous section.

We start from a fixed house allocation instance with applicant set A, house set H, and preference lists $P(a)$ for every applicant $a \in A$. For this instance, we first generate Pareto optimal matchings using the Serial Dictatorship algorithm (code: (B.3)). More precisely, we consider all possible permutations of the applicants. For each permutation, the applicants are processed one by one according to this order, and each applicant receives the most preferred house that is still available, if such a house exists.

By the characterization theorem [ACMM04, Chapter 6, p. 13-14] for Pareto optimal matchings in house allocation, every outcome of Serial Dictatorship is Pareto optimal, and every Pareto optimal matching can be obtained from Serial Dictatorship for some ordering of the applicants. Therefore, by running Serial Dictatorship over all permutations of A and removing duplicate outcomes, we obtain exactly the set of Pareto optimal matchings of the instance.

Each Pareto optimal matching for a given matching M is represented by its incidence vector x^M_{ah} as we defined in Section 3.1.

After generating all Pareto optimal matchings, we obtain a finite set of binary vectors. The Pareto matching polytope of the instance is then defined as

$$P_{\text{POM}} = \text{conv}\{x^M : M \text{ is a Pareto optimal matching}\}.$$

This is the exact convex hull of all Pareto optimal matchings for the given instance.

For small instances, we can compute this convex hull explicitly. Starting from the vertex description, given by the POM vectors, we use compute SageMath to generate the H-description of P_{POM}. In other words, we obtain a description of the form

$$P_{\text{POM}} = \{x : Bx \leqslant d,\ Ex = f\},$$

where the inequalities describe the facets of the polytope and the equations describe its affine hull.

For each instance, we record basic statistics of the resulting polytope. These include the number of variables, the number of distinct Pareto optimal matchings, the dimension of the polytope, the number of equations in the affine hull, and the number of inequalities in the H-description. These statistics give a first indication of the structure of the Pareto matching polytope for the instance under consideration.

The H-description also allows us to test whether a given point belongs to the true convex hull. In particular, if x^* is a feasible solution of the LP relaxation, we can evaluate all equations and inequalities of the H-description at x^*. If one of them is violated, then x^* does not belong to P_{POM}. This gives a direct way to compare the LP relaxation with the actual Pareto matching polytope. This part of the project's source code can be found in (B.2).

The purpose of this computational study is to understand the structure of P_{POM} on small instances. In the final chapter, we use these computations on concrete examples and analyze which inequalities of the true convex hull are violated by fractional solutions of the relaxation.

All the above computations can be found in (B.2).

3.5 Candidate Cutting Inequalities

After relaxing the binary restrictions to $0 \leqslant x_{ah} \leqslant 1, a \in A,\ h \in P(a)$, some of the implications that are enforced by integrality are lost. As a result, the linear relaxation may contain fractional points which do not belong to the convex hull of POM vectors.

In this section we introduce two families of valid inequalities that strengthen the linear relaxation. These inequalities are not claimed to give a complete description of the Pareto-optimal matching polytope. Instead, they are candidate cutting inequalities motivated by the structure of Pareto optimality and by the fractional solutions that appear in the relaxation.

We define

$$y_h = \sum_{a \in Q(h)} x_{ah}.$$

Thus y_h represents whether the house h is assigned. Also, for $a \in A$ and $h \in P(a)$, we define

$$B_a(h) = \{g \in P(a) : g \succ_a h\},$$

that is, the set of houses that agent a prefers to h. Similarly, we define

$$W_a(h) = \{r \in P(a) : h \succ_a r\},$$

the set of houses that are worse than h according to the preferences of a.

3.5.1 First-choice and prefix-cover inequalities

First-choice inequalities. We first consider a simple but important implication of Pareto optimality. Suppose that a house h is the first choice of some agent a. Then, in every Pareto-optimal matching (i.e. in every POM vector), the house h must be assigned. Indeed, if h were unmatched, then either a would be unmatched, contradicting maximality, or a would be matched to a worse house, contradicting trade-in freeness.

Therefore, for every house h that appears as the first choice of at least one agent, the following inequality is valid for the Pareto-optimal matching polytope:

$$\sum_{b \in Q(h)} x_{bh} \geqslant 1.$$

Together with the matching constraint

$$\sum_{b \in Q(h)} x_{bh} \leqslant 1,$$

this implies

$$\sum_{b \in Q(h)} x_{bh} = 1.$$

The reason why this inequality is useful is that the original relaxation does not necessarily enforce it. To see this, fix an agent a whose first choice is h. Since h is the first choice of a, we have $B_a(h) = \varnothing$. The maximality constraint for the pair (a, h) can be written as

$$y_h + \sum_{r \in W_a(h)} x_{ar} \geqslant 1.$$

The trade-in free constraint for the same pair is

$$y_h - \sum_{r \in W_a(h)} x_{ar} \geqslant 0.$$

If we write

$$z_a = \sum_{r \in W_a(h)} x_{ar},$$

then these two constraints become

$$y_h + z_a \geqslant 1,$$

and

$$y_h \geqslant z_a.$$

In the integer formulation, $y_h \in \{0, 1\}$. Hence these two inequalities force $y_h = 1$. In the linear relaxation, however, y_h may be fractional. For example, the values

$$y_h = \frac{1}{2}, \qquad z_a = \frac{1}{2}$$

satisfy both inequalities. Thus the relaxation may allow a first-choice house to be only partially assigned.

The following small instance illustrates this phenomenon. Let

$$A = \{a_1, a_2\}, \qquad H = \{h_1, h_2\},$$

with preference lists

$$P(a_1) = \langle h_1, h_2 \rangle, \qquad P(a_2) = \langle h_2 \rangle.$$

The unique Pareto-optimal matching is

$$M = \{(a_1, h_1), (a_2, h_2)\}.$$

Indeed, the house h_1 is the first choice of a_1, so it must be assigned to a_1, and then maximality forces a_2 to be assigned to h_2.

However, the linear relaxation admits the fractional point

$$x_{11} = \frac{1}{2}, \qquad x_{12} = \frac{1}{2}, \qquad x_{22} = \frac{1}{2},$$

with all other variables equal to zero. The matching constraints are satisfied, since

$$x_{11} + x_{12} = 1,$$

$$x_{22} = \frac{1}{2} \leqslant 1,$$

and

$$x_{12} + x_{22} = 1.$$

The maximality constraints are also satisfied. For example, for the pair (a_1, h_1), the left-hand side is

$$x_{11} + x_{12} = 1,$$

and for the pair (a_2, h_2), the left-hand side is

$$x_{22} + x_{12} = 1.$$

The trade-in constraint for (a_1, h_1) is

$$x_{11} - x_{12} \geqslant 0,$$

which is satisfied with equality. There is no nontrivial coalition cycle in this instance. Hence this point is feasible for the original relaxation.

Nevertheless, it is not contained in the convex hull of POM vectors, because in every POM vector the house h_1 is fully assigned to a_1. The first-choice inequality

$$x_{11} \geqslant 1$$

cuts off this fractional solution.

Dimension. This observation implies an important result concerning the dimension of the polytope. Let F be the set of houses that appear as the first choice of at least one applicant. As shown above, every house $h \in F$ must be assigned in every Pareto-optimal matching. Therefore,

$$\sum_{a \in Q(h)} x_{ah} = 1, \qquad h \in F,$$

is an affine equality for P^{POM}. Each equality contains only variables corresponding to one house, and therefore all these equalities are linearly independent.

Corollary 3.5.1. *Let F be the set of houses that appear as the first choice of at least one applicant. Then*

$$\dim(P^{\mathrm{POM}}) \leqslant |E| - |F|.$$

Prefix-cover inequalities. The previous observation can be generalized. The first-choice inequality is only the special case of a broader family.

Proposition 3.5.2. *For every agent $a \in A$ and every house $h \in P(a)$, the inequality*

$$\sum_{b \in Q(h)} x_{bh} + \sum_{g \in B_a(h)} x_{ag} \geqslant 1$$

is valid for the Pareto-optimal matching polytope.

Proof. Let M be a Pareto-optimal matching, and let x be its incidence vector. We prove that

$$\sum_{b \in Q(h)} x_{bh} + \sum_{g \in B_a(h)} x_{ag} \geqslant 1$$

holds for x.

There are two cases.

First, suppose that the house h is assigned in M. Then

$$\sum_{b \in Q(h)} x_{bh} = 1,$$

and the inequality follows immediately.

Second, suppose that the house h is not assigned in M. Then

$$\sum_{b \in Q(h)} x_{bh} = 0.$$

Since $h \in P(a)$, if agent a were unmatched, then the pair (a, h) could be added to the matching, contradicting maximality. Therefore, a must be matched to some house in $P(a)$.

Moreover, a cannot be matched to a house worse than h. If a were matched to some $r \in W_a(h)$, then h would be an unmatched house preferred by a to the current assignment. This would contradict trade-in freeness.

Thus, when h is unmatched, agent a must be matched to a house $g \in B_a(h)$. Hence

$$\sum_{g \in B_a(h)} x_{ag} = 1.$$

Therefore,

$$\sum_{b \in Q(h)} x_{bh} + \sum_{g \in B_a(h)} x_{ag} = 0 + 1 = 1.$$

The inequality holds in both cases. Since it holds for every POM vector, it is valid for their convex hull. □

We call these inequalities *prefix-cover inequalities*. They say that, for every agent a and every acceptable house h, either the house h must be assigned to some agent, or a must be assigned to a house that appears before h in the preference list of a. When h is the first choice of a, the set $B_a(h)$ is empty, and the inequality reduces exactly to the first-choice house covering inequality.

Cutting off a Fractional Vertex. Consider now the basic relaxation. As previously shown, the vector (3.2) was a fractional vertex of the relaxation. This fractional solution is cut off by the prefix-cover inequality corresponding to the house h_3. Indeed, h_3 is the first choice of both a_3 and a_4. Therefore, the prefix-cover inequality reduces to

$$x_{23} + x_{33} + x_{43} \geqslant 1.$$

However, for the fractional solution above we have

$$x_{23} + x_{33} + x_{43} = 0 + \frac{1}{3} + \frac{1}{3} = \frac{2}{3} < 1.$$

Thus the inequality is violated, and the fractional solution is removed after the prefix-cover cuts are added.

3.5.2 Safe-holder inequalities

The second family of inequalities comes from combining trade-in freeness with coalition freeness.

Suppose that agent a is assigned to a house h, and let $g \in P(a)$ be a house such that

$$g \succ_a h.$$

In a Pareto-optimal matching, the house g must be assigned. Otherwise, agent a could leave h and take the unmatched house g, contradicting trade-in freeness.

However, it is not enough that g is assigned to some agent. The holder of g must be compatible with the fact that a is assigned to h. If g is assigned to an agent b who prefers h to g, then a and b form a two-agent coalition: agent a wants g, while agent b wants h. Such a matching is not coalition-free.

This motivates the following definition. For $a \in A$, $h \in P(a)$, and $g \in B_a(h)$, define the set of safe holders of g with respect to the assignment (a, h) by

$$S(a, h, g) = \{b \in Q(g) \backslash \{a\} : h \notin P(b) \text{ or } g \succ_b h\}.$$

Thus $b \in S(a, h, g)$ if b can hold g without creating a two-agent exchange with a. If $h \notin P(b)$, then b does not find h acceptable, so no such exchange can occur. If $h \in P(b)$, then b is safe precisely when b prefers g to h.

The corresponding inequality is

$$\boxed{x_{ah} \leqslant \sum_{b \in S(a,h,g)} x_{bg}}$$

for every $a \in A$, every $h \in P(a)$, and every $g \in B_a(h)$.

Before proving validity, we give a small example showing why this inequality is useful. Let

$$A = \{a_1, a_2\}, \qquad H = \{h_1, h_2\},$$

with preferences

$$P(a_1) = \langle h_1, h_2 \rangle, \qquad P(a_2) = \langle h_2, h_1 \rangle.$$

The unique Pareto-optimal matching is

$$M = \{(a_1, h_1), (a_2, h_2)\},$$

where both agents receive their first choice. The other perfect matching,

$$M' = \{(a_1, h_2), (a_2, h_1)\},$$

is not Pareto optimal, because the two agents can exchange houses and both improve.

Consider the fractional point

$$x_{11} = x_{12} = x_{21} = x_{22} = \frac{1}{2}.$$

This point satisfies the assignment constraints. It also satisfies the coalition-free inequality corresponding to the two-agent coalition $\{(a_1, h_2), (a_2, h_1)\}$, because

$$x_{12} + x_{21} = \frac{1}{2} + \frac{1}{2} = 1.$$

Thus the coalition inequality is tight, but not violated.

However, this fractional point is not in the convex hull of POM vectors, since the only Pareto-optimal matching is M. The safe-holder inequality identifies this issue. Take

$$a = a_1, \qquad h = h_2, \qquad g = h_1.$$

Agent a_1 prefers h_1 to h_2. The only other agent who can hold h_1 is a_2. But a_2 prefers h_2 to h_1. Therefore, a_2 is not a safe holder of h_1 with respect to the assignment (a_1, h_2). Hence

$$S(a_1, h_2, h_1) = \varnothing.$$

The safe-holder inequality becomes

$$x_{12} \leqslant 0.$$

The fractional point violates this inequality, since

$$x_{12} = \frac{1}{2}.$$

Thus, the safe-holder inequality cuts off a fractional solution that is allowed by the original relaxation.

Proposition 3.5.3. *For every $a \in A$, every $h \in P(a)$, and every $g \in B_a(h)$, the inequality*

$$x_{ah} \leqslant \sum_{b \in S(a,h,g)} x_{bg}$$

is valid for the Pareto-optimal matching polytope.

Proof. Let M be a Pareto-optimal matching, and let x be its incidence vector. We prove that

$$x_{ah} \leqslant \sum_{b \in S(a,h,g)} x_{bg}.$$

If

$$x_{ah} = 0,$$

then the inequality is immediate.

Suppose now that

$$x_{ah} = 1.$$

Thus, in M, agent a is assigned to h. Since $g \in B_a(h)$, we have

$$g \succ_a h.$$

If g were unmatched, agent a could leave h and take g. This would contradict trade-in freeness. Therefore, g must be assigned in M.

Let b be the agent assigned to g. Since a is already assigned to h, we have $b \neq a$. Therefore,

$$b \in Q(g) \backslash \{a\}.$$

We claim that $b \in S(a, h, g)$. Suppose not. Then $h \in P(b)$ and

$$h \succ_b g.$$

But then a and b form a two-agent coalition: agent a, currently assigned to h, prefers g, while agent b, currently assigned to g, prefers h. This contradicts coalition freeness.

Therefore, the holder b of g must belong to $S(a, h, g)$. Hence

$$\sum_{b' \in S(a,h,g)} x_{b'g} = 1.$$

Since $x_{ah} = 1$, we get

$$x_{ah} \leqslant \sum_{b' \in S(a,h,g)} x_{b'g}.$$

Thus, the inequality holds for every Pareto-optimal matching vector, and therefore it is valid for the Pareto-optimal matching polytope. □

The safe-holder inequalities strengthen the relaxation by enforcing, in a linear way, a condition that is automatic in the integer model. In an integral matching, if a receives h and prefers g to h, then g must be assigned to an agent who does not create a two-agent coalition with a. In the linear relaxation, the original constraints may allow g to be assigned fractionally to unsafe holders. The safe-holder inequalities exclude such fractional configurations.

Together, the prefix-cover inequalities and the safe-holder inequalities capture two different implications of Pareto optimality. The prefix-cover inequalities come from maximality and trade-in freeness, while the safe-holder inequalities combine trade-in freeness with coalition freeness. Both families are valid for the Pareto-optimal matching polytope and can be added to the linear relaxation as candidate cutting inequalities.

Chapter 4

Conclusions and Future Work

Appendix A

Implementation Details and Usage Guide

This project was developed in order to study the polytope of Pareto-optimal matchings in the House Allocation problem. The main goal is to compare two objects. The first one is the true convex hull of all Pareto-optimal matching vectors. The second one is the polytope defined by the linear programming relaxation of the integer programming model. By comparing them, we can find fractional solutions of the relaxation and identify valid inequalities that are missing from the relaxation.

The program follows the same general pipeline for every input instance. First, it reads a preference instance from a CSV file. Then it runs the Serial Dictatorship algorithm for all possible orders of the applicants. This gives Pareto-optimal matchings. Since different applicant orders can produce the same matching, duplicate matchings are removed. The remaining unique Pareto-optimal matchings are converted into incidence vectors, called POM vectors. These vectors are then used to construct the convex hull with SageMath. From this convex hull, the program extracts the H-description, namely the equations and inequalities that describe the true Pareto-optimal matching polytope.

After that, the program solves the LP relaxation of the integer programming model. The coalition-free constraints are not added all at once, because there may be exponentially many of them. Instead, they are added by a separation procedure. The program searches for a fractional solution of the relaxation by solving the LP with several random objective functions. If a fractional solution is found, it is compared with the H-description of the true convex hull. The final output shows which equations and inequalities of the true polytope are violated by the fractional relaxation solution.

A.1 Project Structure

The project has the following main folders and files:

```
data/
sd/
lp_relaxation/
polyhedral_analysis/
results/
main.py
README.txt
```

The folder `data/` contains the input instances. Each instance is a CSV file containing the preference lists of the applicants.

The folder `sd/` contains the implementation of Serial Dictatorship. This part generates Pareto-optimal matchings and writes them in vector form.

The folder `lp_relaxation/` contains the implementation of the LP relaxation of the integer programming model. It also contains the separation procedure for the coalition-free constraints.

The folder `polyhedral_analysis/` contains the code that builds the convex hull of the Pareto-optimal matching vectors and extracts its H-description.

The folder `results/` contains the output files. For each input instance, a separate folder is created.

The file `main.py` runs the complete experiment for one selected instance.

A.2 Input Files

Each input instance is stored as a CSV file in the folder `data/`. The first column is called `applicant`. The remaining columns are called `pref1`, `pref2`, and so on. They contain the houses in decreasing order of preference.

For example:

```
applicant,pref1,pref2,pref3,pref4
1,1,2,4,
2,1,3,2,
3,3,5,1,4
4,3,2,5,
5,2,1,,
```

This means that applicant 1 prefers house 1 first, then house 2, then house 4. Empty entries mean that the applicant has no more acceptable houses.

In the current implementation, the input files follow a simple convention: applicants are numbered consecutively as $1, 2, \ldots, n$. The examples used in the experiments are small instances.

A.3 The Main File

The file `main.py` controls the whole experiment. The instance is selected by changing the following line:

```
instance = "instance1"
```

Then the following steps are executed:

1. Run Serial Dictatorship:

```
sd.sd_alg(instance)
```

2. Read the POM vectors:

```
variables, vertices = ip_convex_hull.read_vector_csv(instance)
```

3. Construct the convex hull and extract its H-description:

```
convex_hull = ip_convex_hull.polyhedral_analysis(instance, variables, vertices)
```

4. Solve the LP relaxation and search for a fractional solution:

```
fractional_solution = relaxation.lp_solve(instance)
```

5. If a fractional solution is found, compare it with the H-description of the true polytope:

```
ip_convex_hull.check_fractional_vertex(
instance,
convex_hull,
variables,
fractional_solution
)
```

If no fractional solution is found, the program prints:

```
No fractional vertex was found
```

A.4 Serial Dictatorship Implementation

The file `sd/sd_alg.py` implements the Serial Dictatorship part of the experiment.

The function `read_preferences` reads the CSV file and stores the preferences in a dictionary. For example, if applicant 1 has preference list $1, 2, 4$, then this is stored as:

$$P(1) = [1, 2, 4].$$

The function `serial_dictatorship` considers all permutations of the applicants. For each permutation, the applicants are processed in that order. Each applicant receives the best available house in her preference list. This produces one Pareto-optimal matching for each ordering of the applicants.

Since different permutations may produce the same matching, the function `get_unique_matchings` removes duplicates. The unique matchings are then written to the results folder.

The function `write_pom_vectors` converts each unique matching into an incidence vector. For every acceptable pair (a, h), there is a variable X_{ah}. The value is 1 if applicant a is matched to house h, and 0 otherwise.

The same file also computes simple statistics, such as the number of unique matchings, the minimum and maximum matching size, variables that are never used, and variables that are always used.

A.5 Polyhedral Analysis

The file `polyhedral_analysis/ip_h_description.py` builds the true Pareto-optimal matching polytope.

The function `read_vector_csv` reads the POM vectors from the file:

```
results/<instance>/<instance>_pom_vectors.csv
```

Each row is treated as one vertex of the polytope. The variables are read from the column names, such as `X11`, `X23`, and so on (this only works for single-digit applicants/houses like X34. It breaks for X101, X112, etc. This limitation will be addressed in future versions).

The function `polyhedral_analysis` constructs the convex hull using SageMath:

$$\mathrm{conv}\{x^1, x^2, \ldots, x^k\},$$

where each x^i is a POM vector. SageMath is then used to compute the H-description of this polytope. The H-description consists of equations and inequalities.

The output is written to:

```
results/<instance>/<instance>_polyhedral_analysis.txt
```

This file contains the dimension of the convex hull, the number of vertices, the equations, and the inequalities.

The function `check_fractional_vertex` compares a fractional solution of the LP relaxation with the H-description of the true polytope. It checks which equations and inequalities are satisfied and which are violated.

A.6 LP Relaxation and Separation

The file `lp_relaxation/lp_relaxation_separation.py` implements the LP relaxation of the integer programming model.

For every acceptable pair (a, h), the model has a variable:

$$x_{ah}.$$

In the integer programming model, this variable is binary. In the LP relaxation, it is continuous:

$$0 \leqslant x_{ah} \leqslant 1.$$

The relaxation contains the matching constraints. Each applicant can receive at most one house:

$$\sum_{h \in P(a)} x_{ah} \leqslant 1.$$

Each house can be assigned to at most one applicant:

$$\sum_{a \in Q(h)} x_{ah} \leqslant 1.$$

The model also contains maximality constraints and trade-in-free constraints. These are part of the original model for Pareto-optimal matchings.

The coalition-free constraints are handled differently. They may be exponentially many, so the program does not add all of them at the beginning. Instead, it solves the current relaxation and then searches for a violated coalition constraint. If one is found, it is added to the LP, and the LP is solved again. This is called separation.

The separation procedure builds a directed graph whose vertices are acceptable pairs (a, h). A directed cycle in this graph corresponds to a plausible coalition. For a coalition C, the corresponding constraint is:

$$\sum_{(a,h)\in C} x_{ah} \leqslant |C| - 1.$$

The code searches for violated coalition constraints using shortest path computations. If the current LP solution violates a coalition constraint, that constraint is added to the model.

The program also uses random objective functions in order to search for fractional vertices of the relaxation. If a fractional solution is found, the values are converted to exact rational numbers using SageMath.

Since GLPK returns numerical values as floating-point numbers, the code converts them to exact rational numbers before comparing them with the H-description of the convex hull. This is done with the function `to_QQ`. Values that are very close to 0 or 1 are rounded to exactly 0 or 1, while the remaining values are converted to rational numbers using bounded denominators. This is important because the polyhedral computations in SageMath use the rational field $\mathbb{Q}$. Therefore, after this conversion, equations and inequalities can be checked exactly, avoiding small numerical errors such as 0.999999999 instead of 1.

A.7 Output Files

For every instance, the program creates a folder:

```
results/<instance>/
```

For example, for `instance1`, the output folder is:

```
results/instance1/
```

The main output files are described below.

`<instance>_sd_pareto_matchings.csv` This file contains the result of Serial Dictatorship for every permutation of the applicants.

It has two columns:

- `order`: the order in which the applicants were processed;
- `matching`: the matching produced by that order.

This file may contain duplicate matchings, because different applicant orders can lead to the same matching.

<instance>_unique_pareto_matchings.csv This file contains only the unique Pareto-optimal matchings.

The columns are:

```
A1,A2,A3,...
```

Each row gives one matching. The value in column `Ai` is the house assigned to applicant i. The value -1 means that the applicant is unmatched.

<instance>_pom_vectors.csv This file contains the incidence vectors of the unique Pareto-optimal matchings.

Each column corresponds to a variable X_{ah}. For example, the column `X23` corresponds to the pair $(2, 3)$, meaning applicant 2 matched to house 3.

Each row corresponds to one unique Pareto-optimal matching. A value of 1 means that the pair is used in that matching. An empty cell means 0.

These vectors are the vertices used to construct the convex hull.

<instance>_matching_statistics.txt This file contains simple statistics about the unique Pareto-optimal matchings.

It includes:

- the number of unique matchings;
- the minimum matching size;
- the maximum matching size;
- which matchings have minimum or maximum size;
- variables that are never equal to 1;
- variables that are equal to 1 in every matching.

This file is useful for understanding the structure of the instance before studying the full polytope.

<instance>_polyhedral_analysis.txt This file contains the H-description of the true Pareto-optimal matching polytope.

It includes:

- the dimension of the convex hull;
- the number of vertices;
- the equations of the convex hull;
- the inequalities of the convex hull.

The inequalities in this file are important because they describe the facets or valid inequalities of the true polytope. By comparing them with the LP relaxation, we can see which inequalities are missing from the relaxation.

<instance>_fractional_vertex_hdescription_check.txt This file is created after the LP relaxation finds a fractional solution.

It contains:

- the nonzero coordinates of the fractional solution;
- the number of variables;
- the number of violated equations;
- the number of satisfied equations;
- the number of violated inequalities;
- the number of satisfied inequalities;
- the actual violated inequalities and their residual values.

For an inequality, the program prints the value of:

$$\text{lhs} - \text{rhs}.$$

If this value is negative, then the inequality is violated by the fractional relaxation solution.

This file is one of the most important outputs of the experiment. It shows exactly which inequalities of the true polytope cut off the fractional point.

A.8 How to Run an Experiment

To run an experiment, first choose the instance in `main.py`. For example:

```
instance = "instance1"
```

Then run the main file in an environment where SageMath and GLPK are available. For example:

```
sage -python main.py
```

Alternatively, if the Python interpreter is already configured to use SageMath, the file can be run directly from the IDE.

After the program finishes, the output files are written in:

```
results/<instance>/
```

Appendix B

Source Code

B.1 Linear Programming Relaxation of House Allocation Problem

```
import random as pyrandom
import csv
from sage.all import *
from fractions import Fraction

# convert to exact rationals
def to_QQ(v, max_den=1000000, tol=1e-8):
    vf = float(v)
    if abs(vf) <= tol: return QQ(0)
    if abs(vf - 1) <= tol: return QQ(1)
    return QQ(Fraction(vf).limit_denominator(max_den))

def prefers(agent, rank, h1, h2):
    #True iff agent strictly prefers h1 to h2.
    if h1 not in rank[agent] or h2 not in rank[agent]:
        return False
    return rank[agent][h1] < rank[agent][h2]

#feasibility check - all constraints
#and the cuts already added by separation
def check_exact_feasibility(vals_QQ, V, A, H, P, Q, rank, coalition_constraints=None):
    if coalition_constraints is None:
        coalition_constraints = []
    for v_item in V:
        if vals_QQ[v_item] < QQ(0) or vals_QQ[v_item] > QQ(1): return False
    for a in A:
        if P.get(a) and sum(vals_QQ[(a, h)] for h in P[a]) > QQ(1): return False
    for h in H:
        if Q[h] and sum(vals_QQ[(a, h)] for a in Q[h]) > QQ(1): return False
    for a in A:
        for h in P.get(a, []):
            s1 = sum(vals_QQ[(a, h_p)] for h_p in P[a])
            s2 = sum(vals_QQ[(a_p, h)] for a_p in Q[h] if a_p != a)
            if s1 + s2 < QQ(1): return False
    for a in A:
        for h in P.get(a, []):
            s1 = sum(vals_QQ[(a_p, h)] for a_p in Q[h])
            worse_houses = [h_p for h_p in P[a] if P[a].index(h) < P[a].index(h_p)]
```

```
            s2 = sum(vals_QQ[(a, h_p)] for h_p in worse_houses)
            if s1 - s2 < QQ(0): return False
    for C in coalition_constraints:
        if sum(vals_QQ[(a, h)] for a, h in C) > QQ(len(C) - 1):
            return False
    #prefix-cover strengthening cuts feasibility check
    for a in A:
        prefs = P.get(a, [])
        for i, h in enumerate(prefs):
            better_houses = prefs[:i]
            lhs = (
                sum(vals_QQ[(b, h)] for b in Q[h]) +
                sum(vals_QQ[(a, g)] for g in better_houses)
            )
            if lhs < QQ(1):
                return False
    #safe-holder implication cuts feasibility check
    for a in A:
        prefs = P.get(a, [])
        for i, h in enumerate(prefs):
            better_houses = prefs[:i]
            for g in better_houses:
                safe_holders = []
                for b in Q[g]:
                    if b == a:
                        continue
                    if not prefers(b, rank, h, g):
                        safe_holders.append(b)
                lhs = vals_QQ[(a, h)]
                rhs = sum(vals_QQ[(b, g)] for b in safe_holders)
                if lhs > rhs:
                    return False
    return True

#separation
def separation(sol, G, eps=1e-8):
    """
    Finds a violated coalition-free constraint.
    Input: sol[(a,h)] = current LP value x^*_ah
    Output:
        C = list of vertices [(a,h), ...] forming a violated cycle
        or None if no violated coalition constraint exists.
    """
    #adding weight to each vertex: w(a,h) = 1 - x^*_ah
    w = {v: 1.0 - float(sol[v]) for v in G.vertices()}
    #build weighted directed graph.
    #w(u,v) =  w[v].
    Gw = DiGraph()
    Gw.add_vertices(G.vertices())
    for u, v in G.edge_iterator(labels=False):
        Gw.add_edge(u, v, w[v])
    #all-pairs shortest path distances
    dist = Gw.distance_all_pairs(by_weight=True)
    best_weight = float("inf")
    best_edge = None
    #for every edge (u,v), we check:
    #weight(u,v) + shortest path from v back to u
    for u, v in Gw.edge_iterator(labels=False):
```

```
        if v in dist and u in dist[v]:
            cycle_weight = w[v] + dist[v][u]
            if cycle_weight < best_weight:
                best_weight = cycle_weight
                best_edge = (u, v)
    #no directed cycle exists
    if best_edge is None:
        return None
    #no violated cycle
    if best_weight >= 1.0 - eps:
        return None
    #recover cycle
    u, v = best_edge
    path = Gw.shortest_path(v, u, by_weight=True)
    #cycle is (u, v, ..., u)
    #we take the part (v, ..., u)
    C = [u] + path[:-1]
    return C

def find_fractional_vertex(A, H, P):
    Q = {h: [a for a in A if h in P.get(a, [])] for h in H}
    V = [(a, h) for a in A for h in P.get(a, [])]
    # rank[a][h] = position of house h in agent a's preference list
    rank = {
        a: {h: i for i, h in enumerate(P.get(a, []))}
        for a in A
    }
    #create auxiliary graph
    G = DiGraph()
    G.add_vertices(V)
    for a1, h1 in V:
        for a2, h2 in V:
            if a1 != a2 and h2 in P.get(a1, []):
                if P[a1].index(h2) < P[a1].index(h1):
                    G.add_edge((a1, h1), (a2, h2))
    #lp model
    lp = MixedIntegerLinearProgram(maximization=True, solver="GLPK")
    x = lp.new_variable(real=True, nonnegative=True)
    for a, h in V: lp.set_max(x[a, h], 1.0)
    #add constraints
    for a in A:
        if P.get(a): lp.add_constraint(sum(x[a, h] for h in P[a]) <= 1)
    for h in H:
        if Q[h]: lp.add_constraint(sum(x[a, h] for a in Q[h]) <= 1)

    #============================================================
    #first-choice-houses constraint - added later
    #first_choice_houses = sorted({P[a][0] for a in A if P.get(a)})
    #for h in first_choice_houses:
    #lp.add_constraint(sum(x[a, h] for a in Q[h]) >= 1)
    #prefix-cover constraint - added later, generalization of first-choice-houses
    constraint
    for a in A:
        for i, h in enumerate(P.get(a, [])):
            better_houses = P[a][:i]
            lp.add_constraint(
                sum(x[b, h] for b in Q[h]) +
                sum(x[a, g] for g in better_houses)
```

```
                >= 1
            )
    # #============================================================
    #
    # #============================================================
    # #safe-holder implication cuts
    for a in A:
        prefs = P.get(a, [])
        for i, h in enumerate(prefs):
            better_houses = prefs[:i]
            for g in better_houses:
                safe_holders = []
                for b in Q[g]:
                    if b == a:
                        continue
                    #b is unsafe iff b prefers h to g
                    #so b is safe iff NOT(h preferred to g)
                    if not prefers(b, rank, h, g):
                        safe_holders.append(b)
                lp.add_constraint(
                    x[a, h] <= sum(x[b, g] for b in safe_holders)
                )
    #============================================================

    for a in A:
        for h in P.get(a, []):
            lp.add_constraint(sum(x[a, h_p] for h_p in P[a]) +
                              sum(x[a_p, h] for a_p in Q[h] if a_p != a) >= 1)
    for a in A:
        for h in P.get(a, []):
            worse_houses = [h_p for h_p in P[a] if P[a].index(h) < P[a].index(h_p)]
            lp.add_constraint(sum(x[a_p, h] for a_p in Q[h]) -
                              sum(x[a, h_p] for h_p in worse_houses) >= 0)
    #generate random objective functions
    pyrandom.seed(25)
    coalition_constraints = []
    for trial in range(1, 1001):
        objective_coeffs = {v: pyrandom.randint(-20, 20) for v in V}
        lp.set_objective(sum(QQ(objective_coeffs[v]) * x[v[0], v[1]] for v in V))
        try:
            while(True):
                lp.solve()
                sol = lp.get_values(x)
                C = separation(sol, G)
                if C is None:
                    break
                coalition_constraints.append(C)
                lp.add_constraint(sum(x[a, h] for a, h in C) <= len(C) - 1)
            #rationalize solution
            lp_vals_QQ = {v: to_QQ(sol[v]) for v in V}
            is_fractional = any(QQ(0) < val < QQ(1) for val in lp_vals_QQ.values())
            if is_fractional:
                if check_exact_feasibility(lp_vals_QQ, V, A, H, P, Q, rank,
    coalition_constraints):
                    print("=" * 60)
                    print(f"Found fractional vertex at trial: {trial}")
                    print("-" * 60)
                    print("Objective coefficients:")
```

```
                    for v, c in objective_coeffs.items():
                        if c != 0: print(f"{c} * x_{v}")
                    print("\nFractional Vertex Coordinates:")
                    for v, val_qq in lp_vals_QQ.items():
                        if val_qq > QQ(0):
                            print(f"x_{v} = {val_qq}")
                    return lp_vals_QQ.items()
        except Exception as e:
            print(e)
            continue
    return None

def read_preferences(instance):
    P = {}
    houses = set()
    with open("data/" + instance + ".csv", newline="") as prefcsv:
        reader = csv.DictReader(prefcsv)
        for row in reader:
            applicant = int(row["applicant"])
            prefs = []
            for key, value in row.items():
                if key.startswith("pref") and value != "":
                    house = int(value)
                    prefs.append(house)
                    houses.add(house)
            P[applicant] = prefs
    A = sorted(P.keys())
    H = sorted(houses)
    return A, H, P

def lp_solve(instance):
    A, H, P = read_preferences(instance)
    return find_fractional_vertex(A, H, P)
```

Listing B.1: LP relaxation of house allocation problem, file: lp_relaxation_separation.py

B.2 Convex Hull of Pareto Optimal Vectors

```
import csv
import itertools
from sage.all import *
from pathlib import Path

def read_vector_csv(instance):
    path = Path("results/" + instance + "/" + instance + "_pom_vectors.csv")
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, newline="") as vectors:
        reader = csv.DictReader(vectors)
        columns = reader.fieldnames
        #X11 means applicant 1, house 1
        variables = []
        for col in columns:
            col = col.strip()
            if not col.startswith("X"):
                raise ValueError(f"Invalid column name: {col}")
            a = int(col[1])
            h = int(col[2])
```

```
            variables.append((a, h))
        vertices = []
        for row in reader:
            vec = []
            for col in columns:
                value = row[col].strip()
                if value == "":
                    vec.append(QQ(0))
                else:
                    vec.append(QQ(value))
            vertices.append(vector(QQ, vec))
    return variables, vertices

def variable_name(variable):
    a, h = variable
    return f"X{a}{h}"

def format_term(coef, name):
    coef = QQ(coef)
    if coef == 1:
        return name
    return f"{coef}*{name}"

def format_sum(terms):
    if not terms:
        return "0"
    return " + ".join(terms)

def hrep_sides(hrep, variables):
    coeffs = list(hrep.A())
    const = QQ(hrep.b())
    lhs_terms = []
    rhs_terms = []
    lhs_has_var = False
    rhs_has_var = False
    #sage form: const + coeffs*x >= 0
    #rewrite as: positive terms >= negative terms
    if const > 0:
        lhs_terms.append(str(const))
    elif const < 0:
        rhs_terms.append(str(-const))
    for coef, variable in zip(coeffs, variables):
        coef = QQ(coef)
        name = variable_name(variable)
        if coef > 0:
            lhs_terms.append(format_term(coef, name))
            lhs_has_var = True
        elif coef < 0:
            rhs_terms.append(format_term(-coef, name))
            rhs_has_var = True
    lhs = format_sum(lhs_terms)
    rhs = format_sum(rhs_terms)
    return lhs, rhs, lhs_has_var, rhs_has_var

def readable_hrep(hrep, variables):
    lhs, rhs, lhs_has_var, rhs_has_var = hrep_sides(hrep, variables)
    if hrep.is_equation():
        return f"{lhs} = {rhs}"
```

```
    #1 >= X42 + X43 + X45 -> X42 + X43 + X45 <= 1
    #if not lhs_has_var and rhs != "0":
    #    return f"{rhs} <= {lhs}"
    return f"{lhs} >= {rhs}"

def print_h_description(polytope, variables, instance):
    equations = []
    inequalities = []
    for hrep in polytope.Hrepresentation():
        readable = readable_hrep(hrep, variables)
        if hrep.is_equation():
            equations.append(readable)
        else:
            inequalities.append(readable)
    path = Path("results/" + instance + "/" + instance + "_polyhedral_analysis.txt")
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "a", encoding="utf-8") as poltxt:
        poltxt.write("\nEquations:\n")
        for eq in equations:
            poltxt.write("  " + eq + "\n")
        poltxt.write("\nInequalities:\n")
        for ineq in inequalities:
            poltxt.write("  " + ineq + "\n")

def polyhedral_analysis(instance, variables, vertices):
    convex_hull = Polyhedron(vertices=vertices, base_ring=QQ)
    path = Path("results/" + instance + "/" + instance + "_polyhedral_analysis.txt")
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as poltxt:
        poltxt.write(f"Dimension: {convex_hull.dimension()}\n")
        poltxt.write(f"Number of vertices: {len(vertices)}\n")
    print_h_description(convex_hull, variables, instance)
    return convex_hull

#fractional_vertex = dict_items([((1, 1), 1/7), ((1, 2), 6/7)...
def check_fractional_vertex(
    instance,
    convex_hull,
    variables,
    fractional_vertex,
    tol=QQ(0)
):
    """
    checks a fractional relaxation vertex against the H-description
    of the real POM convex hull.
    """
    fractional_vertex = dict(fractional_vertex)
    #build vector in the same order as variables
    #the polytope coordinates follow the variable order stored in variables,
    #which is the same order as the columns of the pom-vectors csv.
    #the solver output may be unordered and may omit zero-valued variables.
    #therefore, we build x_vec by iterating over variables and retrieving
    #each value from the fractional solution dictionary. the missing entries are 0.
    x_vec = []
    for variable in variables:
        val = fractional_vertex.get(variable, QQ(0))
        x_vec.append(QQ(val))
    x_vec = vector(QQ, x_vec)
```

```
    violated_inequalities = []
    satisfied_inequalities = []
    violated_equations = []
    satisfied_equations = []
    for hrep in convex_hull.Hrepresentation():
        #evaluate each H-description constraint at the fractional relaxation vertex.
        #negative value means a violated facet inequality.
        coeffs = vector(QQ, list(hrep.A()))
        const = QQ(hrep.b())
        value = const + coeffs.dot_product(x_vec)
        readable = readable_hrep(hrep, variables)
        if hrep.is_equation():
            if value != 0:
                violated_equations.append((readable, value))
            else:
                satisfied_equations.append((readable, value))
        else:
            if value < tol:
                violated_inequalities.append((readable, value))
            else:
                satisfied_inequalities.append((readable, value))
    path = Path("results/" + instance + "/" + instance + "
    _fractional_vertex_hdescription_check.txt")
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w") as f:
        f.write("Fractional relaxation vertex\n")
        f.write("=" * 80 + "\n")
        for variable, val in zip(variables, x_vec):
            if val != 0:
                f.write(f"{variable_name(variable)} = {val}\n")
        f.write("\n")
        f.write("Summary\n")
        f.write("=" * 80 + "\n")
        f.write(f"Number of variables: {len(variables)}\n")
        f.write(f"Number of violated equations: {len(violated_equations)}\n")
        f.write(f"Number of satisfied equations: {len(satisfied_equations)}\n")
        f.write(f"Number of violated inequalities: {len(violated_inequalities)}\n")
        f.write(f"Number of satisfied inequalities: {len(satisfied_inequalities)}\n")
        f.write("\n")
        f.write("Violated equations\n")
        f.write("=" * 80 + "\n")
        for readable, value in violated_equations:
            f.write(f"{readable}\n")
            f.write(f"lhs - rhs = {value}\n\n")
        f.write("\n")
        f.write("Satisfied equations\n")
        f.write("=" * 80 + "\n")
        for readable, value in satisfied_equations:
            f.write(f"{readable}\n")
        f.write("\n")
        f.write("Violated inequalities\n")
        f.write("=" * 80 + "\n")
        for readable, value in violated_inequalities:
            f.write(f"{readable}\n")
            f.write(f"lhs - rhs = {value}\n\n")
        f.write("\n")
        f.write("Satisfied inequalities\n")
        f.write("=" * 80 + "\n")
```

```
        for readable, value in satisfied_inequalities:
            f.write(f"{readable}\n")
            f.write(f"lhs - rhs = {value}\n\n")
```

Listing B.2: Description and statistics of the pareto matching polytope, file: ip_h_description.py

B.3 Serial Dictatorship Algorithm

```
#serial dictatorship
#complexity: O(n!)
import csv
import itertools
from pathlib import Path

def read_preferences(filename):
    preferences = {}
    with open(filename, newline="") as prefcsv:
        reader = csv.DictReader(prefcsv)
        for row in reader:
            applicant = int(row["applicant"])
            prefs = []
            for key, value in row.items():
                if key.startswith("pref") and value != "":
                    prefs.append(int(value))
            preferences[applicant] = prefs
    return preferences

def matching_to_tuple(matching, applicants):
    return{
        a: matching.get(a, -1)
        for a in sorted(applicants)
    }

def serial_dictatorship(preferences):
    matchings = {}
    m = len(preferences) #applicants (number of rows)
    applicants = list(range(1, m + 1))
    for perm in itertools.permutations(applicants):
        assigned_houses = set()
        matching = {}
        for applicant in perm:
            for house in preferences[applicant]:
                if house not in assigned_houses:
                    matching[applicant] = house
                    assigned_houses.add(house)
                    break
        matching = matching_to_tuple(matching, applicants)
        matchings[perm] = matching
    return matchings

def all_solutions(matchings, instance):
    #matchings = {perm, dict with matchings}
    n = len(next(iter(matchings.values())))  #applicants
    path = Path("results/" + instance + "/" + instance + "_sd_pareto_matchings.csv")
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", newline="") as sdcsv:
```

```
        writer = csv.writer(sdcsv)
        writer.writerow(["order", "matching"])
        for perm, matching in matchings.items():
            order_str = ",".join(map(str, perm))
            alloc_list = [matching.get(a, -1) for a in perm]
            alloc_str = ",".join(map(str, alloc_list))
            writer.writerow([order_str, alloc_str])

def get_unique_matchings(matchings, m):
    unique = set()
    for matching in matchings.values():
        alloc_tuple = tuple(
            matching.get(a, -1) for a in range(1, m + 1)
        )
        unique.add(alloc_tuple)
    return sorted(unique)

def write_unique_matchings(matchings, m, instance):
    path = Path("results/" + instance + "/" + instance + "_unique_pareto_matchings.csv")
    path.parent.mkdir(parents=True, exist_ok=True)
    unique = get_unique_matchings(matchings, m)
    with open(path, "w", newline="") as uniquecsv:
        writer = csv.writer(uniquecsv)
        writer.writerow([f"A{i}" for i in range(1, m + 1)])
        for alloc in sorted(unique):
            writer.writerow(alloc)
    return unique

def write_pom_vectors(unique_matchings, preferences, instance):
    col = []
    #create Xij columns
    for applicant in sorted(preferences.keys()):
        for house in sorted(set(preferences[applicant])):
            col.append((applicant, house))
    path = Path("results/" + instance + "/" + instance + "_pom_vectors.csv")
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", newline="") as vectorcsv:
        writer = csv.writer(vectorcsv)
        #header
        header = [f"X{applicant}{house}" for applicant, house in col]
        writer.writerow(header)
        #rows
        for matching in unique_matchings:
            row = []
            for applicant, house in col:
                assigned_house = matching[applicant - 1]
                if assigned_house == house:
                    row.append(1)
                else:
                    row.append("")
            writer.writerow(row)

def compute_matching_statistics(unique_matchings, preferences):
    stats = {}
    #number of unique matchings
    stats["num_matchings"] = len(unique_matchings)
    #size of each matching = number of assigned applicants
    matching_sizes = [
```

```
        sum(1 for house in matching if house != -1)
        for matching in unique_matchings
    ]
    min_matching = min(matching_sizes)
    max_matching = max(matching_sizes)
    stats["min_matching"] = min_matching
    stats["max_matching"] = max_matching
    #matching numbers with minimum / maximum size
    stats["min_size_matchings"] = [
        i + 1
        for i, size in enumerate(matching_sizes)
        if size == min_matching
    ]
    stats["max_size_matchings"] = [
        i + 1
        for i, size in enumerate(matching_sizes)
        if size == max_matching
    ]
    #variables Xij that are never equal to 1 in any unique matching
    not_used = []
    for applicant in sorted(preferences.keys()):
        for house in sorted(set(preferences[applicant])):
            used = False
            for matching in unique_matchings:
                if matching[applicant - 1] == house:
                    used = True
                    break
            if not used:
                not_used.append((applicant, house))
    stats["not_used"] = not_used
    #variables Xij that are equal to 1 in every unique matching
    always_used = []
    for applicant in sorted(preferences.keys()):
        for house in sorted(set(preferences[applicant])):
            used_in_all = True
            for matching in unique_matchings:
                if matching[applicant - 1] != house:
                    used_in_all = False
                    break
            if used_in_all:
                always_used.append((applicant, house))
    stats["always_used"] = always_used
    return stats

def write_matching_statistics_txt(stats, instance):
    path = Path("results/" + instance + "/" + instance + "_matching_statistics.txt")
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as stattxt:
        stattxt.write("STATS\n")
        stattxt.write(f"Number of unique matchings: {stats['num_matchings']}\n")
        stattxt.write(
            f"Minimum matching size: {stats['min_matching']} "
            f"(matchings: {', '.join(map(str, stats['min_size_matchings']))})\n"
        )
        stattxt.write(
            f"Maximum matching size: {stats['max_matching']} "
            f"(matchings: {', '.join(map(str, stats['max_size_matchings']))})\n"
        )
```

```
        for applicant, house in stats["not_used"]:
            stattxt.write(f"X{applicant}{house} is never equal to 1 in any matching.\n")
        for applicant, house in stats["always_used"]:
            stattxt.write(f"X{applicant}{house} is equal to 1 in every matching.\n")

def sd_alg(instance):
    input = "data/" + instance + ".csv"
    preferences = read_preferences(input)
    matchings = serial_dictatorship(preferences)
    m = len(preferences) #applicants
    all_solutions(matchings, instance)
    write_unique_matchings(matchings, len(preferences), instance)
    unique_matchings = get_unique_matchings(matchings, m)
    write_pom_vectors(unique_matchings, preferences, instance)
    stats = compute_matching_statistics(unique_matchings, preferences)
    write_matching_statistics_txt(stats, instance)
```

Listing B.3: Serial Dictatorship Algorithm, file: sd_alg.py

B.4 Main Script

```
import polyhedral_analysis.ip_h_description as ip_convex_hull
import sd.sd_alg as sd
import lp_relaxation.lp_relaxation_separation as relaxation

if __name__ == "__main__":
    instance = "instance1"
    sd.sd_alg(instance) #run serial dictatorship algorithm
    variables, vertices = ip_convex_hull.read_vector_csv(instance) #read POM vectors
    convex_hull = ip_convex_hull.polyhedral_analysis(instance, variables, vertices)
    fractional_solution = relaxation.lp_solve(instance)
    if fractional_solution is None:
        print("No fractional vertex was found")
    else:
        ip_convex_hull.check_fractional_vertex(instance, convex_hull, variables,
    fractional_solution)
```

Listing B.4: Main script, file: main.py

References

[ACMM04] David J Abraham, Katarína Cechlárová, David F Manlove, and Kurt Mehlhorn. Pareto optimality in house allocation problems. In *Algorithms and Computation: 15th International Symposium, ISAAC 2004, Proceedings*, pages 3–15. Springer, 2004.

[BBST24] Aaron Bernstein, Joakim Blikstad, Thatchaphol Saranurak, and Ta-Wei Tu. Maximum flow by augmenting paths in $n^{2+o(1)}$ time. In *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 2056–2077, 2024.

[BCK+23] Jan Van Den Brand, Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, Sushant Sachdeva, and Aaron Sidford. A deterministic almost-linear time algorithm for minimum-cost flow. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 503–514, 2023.

[Ber57] Claude Berge. Two theorems in graph theory. *Proceedings of the National Academy of Sciences*, 43(9):842–844, 1957.

[CK24] Julia Chuzhoy and Sanjeev Khanna. Maximum bipartite matching in ⊠2+⊠(1) time via a combinatorial algorithm. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, page 83–94. Association for Computing Machinery, 2024.

[Din70] Yefim A. Dinitz. Algorithm for solution of a problem of maximum flow in networks with power estimation. *Soviet Mathematics Doklady*, 11:1277–1280, 1970.

[Din06] Yefim Dinitz. Dinitz' algorithm: The original version and even's version. In Oded Goldreich, Arnold L. Rosenberg, and Alan L. Selman, editors, *Theoretical Computer Science: Essays in Memory of Shimon Even*, pages 218–240. Springer, Berlin, Heidelberg, 2006.

[EMM15] P Eirinakis, D Magos, and I Mourtos. A model for house allocation. *2015 SCinTE*, page 19, 2015.

[GS62] D. Gale and L. S. Shapley. College admissions and the stability of marriage. *The American Mathematical Monthly*, 69(1):9–15, 1962.

[HK73] John E Hopcroft and Richard M Karp. An n^5/2 algorithm for maximum matchings in bipartite graphs. *SIAM Journal on computing*, 2(4):225–231, 1973.

[MG07] Jiří Matoušek and Bernd Gärtner. *Understanding and Using Linear Programming*. Universitext. Springer Science & Business Media, 2007.

[Pet91] Julius Petersen. Die theorie der regulären graphs, 1891.

[Rot92] Uri G. Rothblum. Characterization of stable matchings as extreme points of a polytope. *Mathematical Programming*, 54(1–3):57–67, 1992.

[Sag26] SageMath. Sagemath glpk backend. https://doc.sagemath.org/html/en/reference/numerical/sage/numerical/backends/glpk_backend.html, 2026. Accessed: 2026-06-11.

[Sch03] Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency, Volume A*, volume 24 of *Algorithms and Combinatorics*. Springer-Verlag, Berlin, Heidelberg, 2003.

[Vat89] John H. Vande Vate. Linear programming brings marital bliss. *Operations Research Letters*, 8(3):147–153, 1989.